

第 8 章 思维的手脚：面向对象编程（上）

到目前为止，你写程序的方式是过程式的：定义一堆函数，函数之间通过参数传递数据。这在小程序里完全够用。但随着项目变大，你会碰到一种烦恼：有些数据和操作这些数据的函数，天然就是绑定在一起的，但你却要分开管理。

比如，你要管理很多学生的信息：名字、分数。每个学生还能做同样的事情：打印自己的信息，计算是否及格。用我们现有的知识，可能会这样写：

```
# 数据

student_names = ["张三", "李四"]

student_scores = [85, 92]

# 操作

def print_student(name, score):

    print(f"{name}: {score}分")
```

数据和操作是分离的。如果再加一个学生，你得同时往两个列表里追加，还不能搞错顺序——这非常容易出 bug。

面向对象编程的解法：把数据和操作它的函数打包成一个整体，叫做类。从类造出来的具体个体，就是对象。这个概念其实你早就在用了——之前用过的字符串、列表、字典，每个都是对象，它们自带方法（比如"hello".upper(), list.append()）。现在，我们要自己设计这样的对象。

这一章和下一章，我们学会如何定义自己的类。你将从一个只会用内置积木的“使用者”，蜕变为能自己设计积木的“创造者”。

8.1 一个真实的问题：管理一百个学生

假设要写一个学生管理系统，每个学生有名字和分数，能判断自己及格了没，还能打印自己的报告。没有类，你得这样：

```
def create_student(name, score):

    return {"name": name, "score": score}

def is_pass(student):

    return student["score"] >= 60

def report(student):

    print(f"{student['name']}: {student['score']}分, {'及格' if is_pass(student)
else '不及格'}")

s1 = create_student("张三", 85)

s2 = create_student("李四", 55)

report(s1)
```

```
report(s2)
```

可以跑，但很别扭：数据和函数是分开的，函数名里必须带 `student_` 前缀来避免冲突。如果还有老师、课程等概念，命名空间会炸。

面向对象的思路：

```
class Student:

    def __init__(self, name, score):

        self.name = name

        self.score = score

    def is_pass(self):

        return self.score >= 60

    def report(self):

        print(f"{self.name}: {self.score}分, {'及格' if self.is_pass() else '不及格'}")

s1 = Student("张三", 85)

s2 = Student("李四", 55)

s1.report()

s2.report()
```

数据和操作被封装进 `Student` 类。`s1` 和 `s2` 是两个独立的实例，每个都带着自己的数据，知道自己能干什么。

8.2 类与实例：造人的模板和具体的人

类就是一个模板（蓝图），实例就是用模板造出来的具体个体。类是抽象的“学生”概念，实例是张三、李四这些具体的学生。

定义类的语法：

```
class 类名:

    """文档字符串"""

    缩进的方法定义（就是函数在类里叫方法）
```

类名按约定用大驼峰命名（CapWords），比如 `Student`、`BankAccount`、`ShoppingCart`。

创建实例就是调用类，像调用函数一样：

```
s = Student() # 造出一个 Student 对象，贴标签 s
```

现在，我们来解剖第一个完全体的类。

8.3 __init__方法：出生的那一刻

在类里，有一个特殊方法__init__（注意是双下划线），它在实例被创建时自动调用。用来做初始化——给每个实例贴上它专属的数据。

```
class Student:

    def __init__(self, name, score):

        self.name = name    # 实例属性

        self.score = score
```

当我们写 `s1 = Student("张三", 85)` 时，背后发生了什么？

1. Python 创建一个空白的 Student 对象。
2. 自动调用 `__init__(self, name, score)`，把 `s1` 传给 `self`，"张三"传给 `name`，85 传给 `score`。
3. 在方法内部，`self.name = name` 就把“张三”这个标签贴到了 `self`（也就是 `s1`）的 `name` 属性上。
4. 最后，这个初始化好的对象返回给 `s1`。

新手坑位：__init__不是构造器。它负责初始化一个已经存在的对象，而真正创建对象的是__new__，我们基本不碰。只需记住：__init__是给刚出生的对象穿衣服。

8.4 self：我自己的指代

self 是类里所有方法的第一参数。它是习俗，不是关键字——你叫它 `this` 也行，但 Python 社区强烈约定成俗用 `self`。

self 代表调用这个方法的那个实例本身。当你写 `s1.report()` 时，Python 实际执行的是 `Student.report(s1)`。self 就是 `s1`。

所以，在方法内部，你通过 `self.xxx` 来访问或修改当前实例的数据。没有 `self`，方法就不知道该操作谁的数据。

新手坑位：忘了写 `self` 作为第一个参数。如果你定义方法时不写 `self`，调用时会报 `TypeError: xxx() takes 0 positional arguments but 1 was given`。这个错误信息新手读不懂，其实意思是“实例方法被调用时自动传入实例本身，但你的方法签名没接受它”。务必每个实例方法第一个参数写 `self`。后面学静态方法时会打破这个规则，但现在请严格遵守。

8.5 实例属性和方法

实例属性就是 `self.xxx` 这样的变量，属于每个实例自己。你可以随时动态添加属性，但强烈建议都在__init__里初始化好，这样代码清晰：

```
class Dog:

    def __init__(self, name):

        self.name = name

d = Dog("旺财")
```

```
print(d.name) # 旺财
```

方法就是类里面定义的函数。调用时通过实例：`d.bark()`。

```
class Dog:

    def __init__(self, name):

        self.name = name

    def bark(self):

        print(f"{self.name}: 汪汪!")

d = Dog("旺财")

d.bark() # 旺财: 汪汪!
```

8.6 从过程到对象：重写词频统计

在第7章，我们用函数写了词频统计。现在把它重构成面向对象版本，看看类的威力——尤其是数据状态的管理。

```
import string

class WordCounter:

    """词频统计器"""

    def __init__(self):

        self.word_counts = {} # 初始化空字典

    def process_line(self, line):

        """处理一行文本"""

        line = line.lower()

        line = line.translate(str.maketrans("", "", string.punctuation))

        words = line.split()

        for word in words:

            self.word_counts[word] = self.word_counts.get(word, 0) + 1

    def load_file(self, filename):

        """从文件读取并统计"""

        try:

            with open(filename, "r", encoding="utf-8") as f:

                for line in f:

                    self.process_line(line)

        except FileNotFoundError:
```



```

        print(f"文件 {filename} 不存在")

    def top_words(self, n=20):
        """返回频率最高的 n 个词"""

        sorted_items = sorted(self.word_counts.items(), key=lambda x: x[1],
reverse=True)

        return sorted_items[:n]

    def clear(self):
        """重置统计"""

        self.word_counts.clear()

wc = WordCounter()
wc.load_file("novel.txt")
for word, freq in wc.top_words(10):
    print(f"{word}: {freq}")

```

和过程式版本比，好处显而易见：

- 1.WordCounter 对象内部维护状态（self.word_counts），不用传来传去。
- 2.你可以创建多个 WordCounter 实例，各自独立统计不同文件。
- 3.添加新功能（比如 clear()）只需加一个方法，不改已有代码。

8.7 一个容易犯的错：类属性 vs 实例属性

直接在类体里写（不在方法里）的变量是类属性，被所有实例共享。这往往不是你想要的效果。

```

class Student:
    school = "北京大学" # 类属性，所有实例共享

s1 = Student()
s2 = Student()

print(s1.school) # 北京大学
print(s2.school) # 北京大学

Student.school = "清华大学"

print(s1.school) # 清华大学
print(s2.school) # 清华大学 （全变了）

```

如果你想每个学生有不同的学校，必须在__init__里用 self.school = ...。类属性的一个典型用途是常量或计数器：

```

class Student:

```

```

count = 0    # 统计创建了多少学生

def __init__(self, name):

    self.name = name

    Student.count += 1    # 每创建一个实例，类属性加 1

print(Student.count)    # 0

s1 = Student("张三")

s2 = Student("李四")

print(Student.count)    # 2

```

规则：如果属性值每个实例不一样，用 `self.xxx` 在 `__init__` 里初始化。如果所有实例共享同一个值，且修改会影响全局，考虑用类属性。

8.8 特殊方法：让你的对象像内置对象

Python 里 `len(lst)`、`str(x)`、`a == b` 这些操作能工作，是因为背后调用了对象的特殊方法（双下划线方法）。你也可以实现它们，让自己的对象跟内置类型一样自然。

最常见的几个：

```

class Student:

    def __init__(self, name, score):

        self.name = name

        self.score = score

    def __str__(self):

        """print() 或 str() 时调用，给人看的"""

        return f"{self.name}, {self.score}分"

    def __repr__(self):

        """在交互模式直接输对象时调用，给开发者看的，通常返回能重新创建对象的字符串"""

        return f"Student('{self.name}', {self.score})"

    def __eq__(self, other):

        """== 比较"""

        if not isinstance(other, Student):

            return False

        return self.name == other.name and self.score == other.score

s1 = Student("张三", 85)

s2 = Student("张三", 85)

print(s1)          # 张三, 85 分

```

```
print(repr(s1))    # Student('张三', 85)
print(s1 == s2)    # True （没有 __eq__ 时默认比较是否为同一个对象，即 s1 is s2）
```

实现`__str__`和`__eq__`，让你的对象打印友好，比较合理。将来在数据类里你还会见到更多特殊方法，但这三个是入门首推。

8.9 实战：一个简易银行账户

我们来实现一个 `BankAccount` 类，支持存钱、取钱、打印余额，并防止取钱超额。

```
class BankAccount:
    """银行账户"""

    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        """存款"""
        if amount <= 0:
            print("存款金额必须为正数")
            return
        self.balance += amount
        print(f"存入 {amount} 元，余额 {self.balance} 元")

    def withdraw(self, amount):
        """取款"""
        if amount <= 0:
            print("取款金额必须为正数")
            return
        if amount > self.balance:
            print(f"余额不足（当前余额 {self.balance} 元）")
            return
        self.balance -= amount
        print(f"取出 {amount} 元，余额 {self.balance} 元")

    def __str__(self):
        return f"账户 [{self.owner}], 余额: {self.balance} 元"

# 使用
```

```
acc = BankAccount("张三", 1000)
acc.deposit(500)
acc.withdraw(2000)
acc.withdraw(300)
print(acc)
```

输出：

```
存入 500 元，余额 1500 元
余额不足（当前余额 1500 元）
取出 300 元，余额 1200 元
账户[张三]，余额：1200 元
```

再看一下封装的好处：你完全可以把 `balance` 这个属性直接暴露给外部，但那样用户可能直接写 `acc.balance = 99999` 绕过验证。Python 没有真正的私有属性，但是有一个约定：用一个下划线开头的名字表示“私有，别碰”。例如 `self._balance`。这只是约定，但大家都遵守。后面学 `property` 装饰器可以优雅地控制访问，那是进阶内容，现在先用 `self.balance` 有意识地不去直接修改就好。

8.10 总结与下一步

这一章，你从过程式的思维方式，迈入了面向对象的世界：

类是蓝图，实例是具体的产物。

`__init__` 初始化实例属性。

`self` 指代实例本身，通过它访问属性和方法。

方法就是属于类的函数。

特殊方法让你的对象像内置类型一样打印、比较。

现在，你设计一个类，就是在创造一种新的积木。下一章，我们将深入学习继承与组合——让类之间产生“父子”和“包含”关系，减少重复代码，构建更强大的系统。

练习

1. 定义一个 `Rectangle` 类，属性有长 `length` 和宽 `width`，方法 `area()` 返回面积，`perimeter()` 返回周长。

2. 定义一个 `Circle` 类，属性 `radius`，方法 `area()` 和 `circumference()`（周长）。圆周率用 `3.14` 或 `math.pi`。

3. 给第 1 题的 `Rectangle` 添加 `__str__` 方法，打印“矩形(长 x 宽)”；添加 `__eq__` 方法，面积相等即视为相等。

4. 用面向对象实现一个简单的待办事项类 `ToDoList`，内部用列表存事项，支持 `add(task)`、`remove(task)`（如果不存在提示）、`show_all()`，以及 `clear()`。

5. 下面的代码有什么问题？如何修正？

```
class Person:

    def __init__(name, age):

        self.name = name

        self.age = age
```

练习参考答案

练习 1

```
class Rectangle:

    def __init__(self, length, width):

        self.length = length

        self.width = width

    def area(self):

        return self.length * self.width

    def perimeter(self):

        return 2 * (self.length + self.width)

rect = Rectangle(5, 3)

print(rect.area())      # 15

print(rect.perimeter()) # 16
```

练习 2

```
import math

class Circle:

    def __init__(self, radius):

        self.radius = radius

    def area(self):

        return math.pi * self.radius ** 2

    def circumference(self):

        return 2 * math.pi * self.radius

c = Circle(5)

print(c.area())          # 78.53981633974483

print(c.circumference()) # 31.41592653589793
```

练习 3

```
class Rectangle:
```

```

def __init__(self, length, width):
    self.length = length
    self.width = width

def area(self):
    return self.length * self.width

def perimeter(self):
    return 2 * (self.length + self.width)

def __str__(self):
    return f"矩形({self.length}x{self.width})"

def __eq__(self, other):
    if not isinstance(other, Rectangle):
        return False
    return self.area() == other.area()

r1 = Rectangle(4, 5)
r2 = Rectangle(2, 10)

print(r1)          # 矩形(4x5)
print(r1 == r2)    # True （面积都是 20）

```

练习 4

```

class TodoList:
    def __init__(self):
        self.tasks = []

    def add(self, task):
        self.tasks.append(task)
        print(f"已添加: {task}")

    def remove(self, task):
        if task in self.tasks:
            self.tasks.remove(task)
            print(f"已删除: {task}")
        else:
            print(f"事项 '{task}' 不存在")

```

```

def show_all(self):
    if not self.tasks:
        print("待办列表为空")
    else:
        for i, task in enumerate(self.tasks, 1):
            print(f"{i}. {task}")

def clear(self):
    self.tasks.clear()
    print("已清空所有事项")

todo = TodoList()
todo.add("买苹果")
todo.add("写作业")
todo.show_all()
todo.remove("买苹果")
todo.show_all()
todo.clear()
todo.show_all()

```

练习 5

缺少 `self` 参数。`__init__` 必须至少有一个参数 `self`：

```

class Person:

    def __init__(self, name, age):

        self.name = name

        self.age = age

```

否则调用 `Person("张三", 20)` 时，"张三" 会传给 `name` 形参，20 传给 `age`，但 Python 自动传入的实例对象无处安放，导致 `TypeError`。

第 9 章 继承与组合：面向对象编程（下）

上一章你学会了定义自己的类。但程序里的类往往不是孤岛——它们之间有千丝万缕的关系。比如 `Student` 和 `Teacher` 都是 `Person`，它们共享名字、年龄这些特征，但各自又有不同的行为。如果把共享的代码在两个类里各写一遍，一旦要改就得改两处。

这就是继承要解决的问题：用一个“父类”装共享的代码，让“子类”自动拥有它们，子

类只需写自己独特的部分。但继承不是万能的，用不好会造出非常脆弱的代码。很多时候，组合——在一个类里包含另一个类的实例——才是更好的选择。

这一章，我们把两种方式都讲透，并让你能判断什么时候该用哪个。

9.1 一个真实的问题：鸭子、企鹅和鸟

假设你要模拟一个动物园。有 Duck（鸭子），它会游泳、会叫；有 Penguin（企鹅），它会游泳但不会飞，叫声不同；以后可能还有 Sparrow（麻雀）、Ostrich（鸵鸟）。

没有继承，每个类从头写到尾，swim()方法在好几个类里完全一样，复制粘贴。一旦要改游泳的实现，得改好几处。软件工程里，重复是万恶之源。

继承的解法：定义一个基类 Bird，把共性的东西放进去：

```
class Bird:

    def __init__(self, name):

        self.name = name

    def swim(self):

        print(f"{self.name} 在水中游泳")
```

然后让 Duck 和 Penguin 继承 Bird，自动获得 swim 能力，只写各自不同的部分。

9.2 继承的语法：子类获得父类的一切

继承语法：在类名后用括号指定父类。

```
class Duck(Bird):

    def quack(self):

        print(f"{self.name}: 嘎嘎！")

class Penguin(Bird):

    def honk(self):

        print(f"{self.name}: 咯咯！")
```

Duck 和 Penguin 是子类，Bird 是父类（超类）。子类自动继承父类所有属性和方法：

```
d = Duck("唐老鸭")

d.swim() # 唐老鸭 在水中游泳 （继承来的）

d.quack() # 唐老鸭: 嘎嘎！ （自己的）
```

9.3 子类扩展__init__: super()

如果子类需要在初始化时加自己的属性，它可以定义自己的__init__，并在里面调用父类的__init__——用 super()。

```
class Bird:
```



```

    def __init__(self, name):
        self.name = name

class Duck(Bird):
    def __init__(self, name, color):
        super().__init__(name)    # 调用父类 __init__, 设置 name
        self.color = color        # 子类新增属性

d = Duck("唐老鸭", "白色")
print(d.name)    # 唐老鸭
print(d.color)   # 白色

```

`super().__init__(name)`的意思是：找到当前类的父类（这里是 `Bird`），调用它的 `__init__` 方法，把 `name` 传进去。

新手坑位：忘记 `super().__init__()`。

如果子类定义了 `__init__` 但没调用 `super().__init__()`，父类的初始化代码根本不会执行，父类的属性不会被设置。这是继承里最常见的 bug 之一。

```

# 错误示范

class Duck(Bird):
    def __init__(self, name, color):
        self.color = color    # 忘了调用 super().__init__()

d = Duck("唐老鸭", "白色")
print(d.color)    # 白色
print(d.name)     # AttributeError: 'Duck' object has no attribute 'name'

```

9.4 方法重写：子类覆盖父类行为

有时候父类提供了默认行为，但某个子类需要不一样。子类可以定义一个同名方法来重写它：

```

class Bird:
    def __init__(self, name):
        self.name = name

    def move(self):
        print(f"{self.name} 会走路")

class Penguin(Bird):
    def move(self):
        print(f"{self.name} 摇摇摆摆地走路")

```

```
class Eagle(Bird):
    def move(self):
        print(f"{self.name} 展翅高飞")

birds = [Bird("普通鸟"), Penguin("皮皮"), Eagle("老鹰")]

for b in birds:
    b.move()
```

输出：

```
普通鸟 会走路
皮皮 摇摇摆摆地走路
老鹰 展翅高飞
```

同一个 `move()` 方法名，不同子类有不同的表现。这就是多态——一个接口，多种形态。`for` 循环不需要知道 `b` 是哪种鸟，只要它有 `move()` 方法就行。

9.5 “是一个” vs “有一个”：继承与组合的分界线

到这里你可能会觉得继承很强大，然后什么都想继承。但继承有个著名陷阱。

假设你给 `Bird` 类加一个 `fly()` 方法：

```
class Bird:
    def fly(self):
        print(f"{self.name} 起飞！")
```

然后让 `Penguin` 也继承它。可企鹅不会飞。你被迫在 `Penguin` 里重写 `fly()`，让它要么什么都不做，要么抛异常：

```
class Penguin(Bird):
    def fly(self):
        print(f"{self.name} 不会飞")
```

这已经有点奇怪了。随着鸟类增多——鸵鸟、几维鸟、渡渡鸟——不会飞的鸟越来越多，`Bird` 的 `fly()` 变得不再通用。你当初为什么把它放在 `Bird` 里？

这就是经典的继承滥用：因为图方便，把不是所有子类都具备的能力塞进父类，最后父类臃肿不堪，子类也被迫重写一堆不必要的方法。

设计原则：继承描述“是一个”关系，组合描述“有一个”关系。

继承：`Duck` 是一个 `Bird`，`Penguin` 是一个 `Bird`。共享核心特征时用继承。

组合：`Duck` 有一个 `Wing`，有一个 `Beak`。功能模块化时用组合。

对于飞行能力，更好的设计是用组合——把会飞的能力抽象成一个独立的“模块”，需要飞的鸟带上它：

```

class FlyBehavior:
    def fly(self, name):
        print(f"{name} 展翅高飞")

class CannotFly:
    def fly(self, name):
        print(f"{name} 不会飞")

class Bird:
    def __init__(self, name, fly_behavior):
        self.name = name
        self.fly_behavior = fly_behavior    # 组合：把飞行行为注入进来
    def perform_fly(self):
        self.fly_behavior.fly(self.name)

eagle = Bird("老鹰", FlyBehavior())
penguin = Bird("企鹅", CannotFly())

eagle.perform_fly()    # 老鹰 展翅高飞
penguin.perform_fly() # 企鹅 不会飞

```

现在，飞行能力是独立的对象，可以被“注入”到任何鸟里。你可以组合出无穷无尽的行为，而不用修改 Bird 类。这就是策略模式的思想，是所有大型框架的基石。仅从这个小例子，你已经触达了设计模式的门槛。

9.6 继承不是写代码最快的，而是封装变化最少的

如果你只有两三个类，直接用继承也没问题。但当层级超过两三层，或者你发现父类里出现了“有的子类不需要的方法”时，立刻停下来，考虑组合。

继承和组合的选择清单：

用继承：所有子类共享同一个基类行为，子类没有特殊情况。例如“所有动物都会呼吸”，这放父类没问题。

用组合：不同对象需要不同功能“插件”，功能可能会变化，或者并非所有子类都拥有。例如飞行、叫声、游泳。

何时都不应该用：仅仅为了少写几行代码而强行继承一个无关的类。

如果是初学，现在脑子里有一个大概的印象即可。读完这本书，当你开始写第一个超过 500 行的程序时，你会切身体会到这些原则的重要。

9.7 Python 支持多继承，但要极其克制

Python 支持一个类继承多个父类：

```
class A:
    def method_a(self): print("A")

class B:
    def method_b(self): print("B")

class C(A, B):
    pass

c = C()

c.method_a() # A
c.method_b() # B
```

多继承会让属性查找顺序变得复杂（Python 用 C3 线性化算法确定方法解析顺序，`Class.__mro__`可以查看）。大多数时候你不需要多继承。如果感觉自己需要的功能分散在多个类里，先考虑组合是否能解决问题。

9.8 实战：重构学生管理系统

我们在第 8 章写了一个 `Student` 类。现在系统里有 `Teacher`——也有名字，但分数变成了工资，还要一个独特的 `teach()` 方法。

先用继承重构，再对比组合方案。

继承方案

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def introduce(self):
        return f"我叫{self.name}, {self.age}岁"

class Student(Person):
    def __init__(self, name, age, score):
        super().__init__(name, age)
        self.score = score
    def introduce(self):
        return super().introduce() + f", 分数: {self.score}"
```

```

class Teacher(Person):
    def __init__(self, name, age, salary):
        super().__init__(name, age)
        self.salary = salary
    def teach(self):
        print(f"{self.name} 老师正在讲课")
    def introduce(self):
        return super().introduce() + f", 工资: {self.salary}元"

s = Student("张三", 18, 88)
t = Teacher("李老师", 35, 8000)
print(s.introduce())
print(t.introduce())
t.teach()

```

这个继承用得合理：Student 和 Teacher 都是 Person，共享 name 和 age，各自加属性。

组合方案（更灵活）

如果未来还有 Staff（职员）、Parent（家长），每个人“角色”可能不同但核心数据一样，这就适合组合：

```

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

class Role:
    def describe(self):
        return ""

class StudentRole(Role):
    def __init__(self, score):
        self.score = score
    def describe(self):
        return f"学生, 分数: {self.score}"

class TeacherRole(Role):
    def __init__(self, salary):
        self.salary = salary

```

```

def describe(self):
    return f"教师, 工资: {self.salary}元"

def teach(self):
    print("讲课中...")
# 一个人可以挂多个角色

class PersonWithRole:
    def __init__(self, name, age, role):
        self.name = name
        self.age = age
        self.role = role    # 组合

    def introduce(self):
        return f"我叫{self.name}, {self.age}岁, {self.role.describe()}"

p1 = PersonWithRole("张三", 18, StudentRole(88))
p2 = PersonWithRole("李老师", 35, TeacherRole(8000))

print(p1.introduce())
print(p2.introduce())

```

组合方案优势明显：同一个人以后既是学生又是助教，加上两个角色即可，不需要新建奇怪的 StudentTeacher 类。这就是组合带来的弹性。

9.9 总结与下一步

这一章，你看到了对象之间两种核心关系：

继承：“是一个”，子类共享父类核心特征。语法上用 `class Sub(Parent)`。

组合：“有一个”，对象包含其他对象作为属性。更灵活，更容易扩展。

`super()`让子类调用父类方法，尤其在 `__init__` 中。

多态让不同子类可以对同一方法名做出不同响应。

继承有陷阱，当“不是所有子类都需要”时，用组合拆分行为。

现在，你设计对象的能力达到了一个新高度。下一章，我们将学习模块与包——把你的代码组织成文件，摆脱单文件脚本，迈入真正项目的世界。你写的类会变成可以复用的模块，被任何项目导入。

练习

1. 定义一个 Vehicle 基类，有 brand 和 speed 属性，方法 info() 打印信息。子类 Car 加属性 doors 并重写 info()，子类 Bicycle 加属性 type（山地/公路）。

2. 用 `super()` 实现第 1 题的 `__init__`。

3.解释：如果给 Vehicle 加一个 fly()方法，而 Car 和 Bicycle 都不会飞，为什么不合适？你会怎么改？

4.实现一个 Battery 类，属性 capacity（容量），方法 charge()打印“充电中”。然后让一个 ElectricCar 类用组合方式拥有一个 Battery 实例，而不是继承 Battery。

5.查看 list 类的 __mro__ 属性（用 print(list.__mro__)），说出你看到了什么，它告诉你什么信息。

练习参考答案

练习 1 & 2

```
class Vehicle:

    def __init__(self, brand, speed):

        self.brand = brand

        self.speed = speed

    def info(self):

        return f"{self.brand}, 最高时速 {self.speed} km/h"

class Car(Vehicle):

    def __init__(self, brand, speed, doors):

        super().__init__(brand, speed)

        self.doors = doors

    def info(self):

        return super().info() + f", {self.doors} 门轿车"

class Bicycle(Vehicle):

    def __init__(self, brand, speed, bike_type):

        super().__init__(brand, speed)

        self.type = bike_type

    def info(self):

        return super().info() + f", {self.type}自行车"

c = Car("丰田", 180, 4)

b = Bicycle("捷安特", 30, "山地")

print(c.info())

print(b.info())
```

练习 3

把 fly()放在 Vehicle 里不合适，因为不是所有交通工具都会飞。Car 和 Bicycle 被迫重写或忽

略它，这违反了继承的“is-a”原则。更好的做法是用组合：定义一个 FlyBehavior 类，只让会飞的交通工具（比如 Airplane）包含它。

练习 4

```
class Battery:
    def __init__(self, capacity):
        self.capacity = capacity

    def charge(self):
        print(f"电池 ({self.capacity}kWh) 充电中...")

class ElectricCar:
    def __init__(self, brand, capacity):
        self.brand = brand

        self.battery = Battery(capacity)  # 组合，不是继承

    def charge(self):
        print(f"{self.brand} 开始充电")

        self.battery.charge()

tesla = ElectricCar("Tesla", 100)
tesla.charge()
```

练习 5

```
print(list.__mro__)

# (<class 'list'>, <class 'object'>)
```

这告诉你 list 继承自 object（object 是所有类的终极基类）。方法解析顺序是从左到右：先在 list 里找方法，找不到再到 object 里找。如果有多继承，__mro__ 会给出完整查找路径。

第 10 章 模块与包：拥抱开源世界

你已经能写类，能设计对象。但所有代码还挤在一个.py 文件或一个笔记本里。当程序超过几百行，在一个文件里翻来翻去会越来越痛苦。你开始想：能不能把不同功能的代码拆到不同文件里？能不能把写好的工具给别的项目用？能不能用别人写好的现成轮子，而不是自己造？

这就是模块与包要解决的问题。这一章，你将从“写脚本”跨越到“构建项目”。你会学会引用标准库、安装第三方库、组织自己的代码文件结构。这是从“会写 Python 语法”到“能用 Python 做项目”的关键一步。

10.1 一个真实的问题：代码越来越长了

翻一翻你前面章节写的代码。如果把它们全塞进一个文件，已经几百行了。再过几章，一个程序可能上千行。在一个巨长的文件里找到某个函数，就像在一堆杂物里找钥匙。

更麻烦的是：你写了一个好用的 WordCounter 类，想在另一个项目里用。复制粘贴？那以后改了 bug 得同步改好几份。这是所有语言都要解决的问题。

答案是：把代码拆成模块。一个.py 文件就是一个模块。你其实已经在用模块了——import math、import string 就是导入 Python 自带的标准库模块。

现在，我们要学怎么自己造模块、怎么组织它们、怎么用别人造好的。

10.2 import 的基础：把别人的代码拿过来用

import 就是把一个模块加载到当前程序，让里面的函数和类为你所用。

```
import math

print(math.pi)          # 3.141592653589793

print(math.sqrt(16))    # 4.0
```

import math 做了两件事：

- 1.找到 math 模块（它是一个.py 文件，或者 C 扩展）。
- 2.在当前命名空间创建名字 math，指向这个模块对象。

然后你用 math.xxx 访问里面的内容。这个 math.前缀是命名空间，防止同名冲突。

from import：只拿需要的东西

如果你只用 pi，不想每次写 math.pi，可以：

```
from math import pi, sqrt

print(pi)

print(sqrt(16))
```

这直接把 `pi` 和 `sqrt` 放进当前命名空间，省了前缀。

新手坑位：`from math import *`不推荐。

星号导入会把这个模块里所有未以下划线开头的名字全部引入当前空间，你根本不知道引入了什么，可能会覆盖你已有的同名变量，非常难排查。除非是在临时实验的交互模式里，否则不要用。

import as: 起别名

有些模块名字长，或你想避免冲突，可以用 `as` 给模块起个小名：

```
import numpy as np
import pandas as pd
```

这是数据科学领域的约定俗成，读到 `np.array` 你立刻知道是 `numpy` 的数组。

10.3 Python 找模块的顺序：sys.path

当你 `import something` 时，Python 按以下顺序搜索：

- 1.当前脚本所在的目录
- 2.环境变量 `PYTHONPATH` 里的路径
- 3.Python 安装时带的标准库路径
- 4.第三方包安装的 `site-packages` 目录

这些路径全在一个列表 `sys.path` 里。你可以随时查看：

```
import sys
print(sys.path)
```

新手坑位：请不要给自己写的模块命名和标准库一样的名字。

比如你建了一个 `math.py`，然后 `import math`，Python 会先找到你的 `math.py` 而不是标准库，然后报一堆奇怪的错。常见踩坑名字：`math.py`、`test.py`、`email.py`、`csv.py`。起模块名之前去查一下有没有同名标准库或著名第三方库。

10.4 写自己的模块：你的.py 文件就是模块

假设你有个 `utils.py`，内容如下：

```
# utils.py
def add(a, b):
    return a + b
def multiply(a, b):
    return a * b
PI = 3.14159
```

在同一个目录下，另一个脚本 `main.py`：

```
# main.py

import utils

print(utils.add(3, 4))      # 7

print(utils.PI)            # 3.14159
```

就这么简单。任何.py 文件都可以被导入。

但有一个关键问题：当你 `import utils` 时，Python 会执行一遍 `utils.py` 里的所有代码（定义函数、定义变量等）。这意味着如果 `utils.py` 里有直接运行的语句（比如 `print("hello")`），它会在导入时执行一次——通常这不是你想要的。

10.5 if `__name__ == "__main__"`: 区分导入和直接运行

每个 Python 文件都有个隐藏属性 `__name__`：

如果这个文件被直接运行（`python utils.py`），`__name__` 的值是 `"__main__"`。

如果这个文件被导入（`import utils`），`__name__` 的值是 `"utils"`（模块名）。

利用这个机制，你可以写测试代码或示例代码，确保只在直接运行时执行，导入时不打扰：

```
# utils.py

def add(a, b):

    return a + b

def multiply(a, b):

    return a * b

if __name__ == "__main__":

    # 这部分只在直接运行 utils.py 时执行

    print("测试 add:", add(3, 5))

    print("测试 multiply:", multiply(3, 5))
```

当你 `import utils` 时，这两个 `print` 不会执行。当你 `python utils.py` 时，它们才会跑。

这是 Python 脚本的标准写法。从现在开始，你写的每个模块，应该在末尾加上 `if __name__ == "__main__":` 块存放测试或演示代码。

10.6 包：把模块放进文件夹

如果你有三个模块 `utils.py`、`models.py`、`services.py`，它们都堆在根目录，还是乱。把它们放进一个文件夹，就是包。

一个包就是包含 `__init__.py` 文件的文件夹。这个文件可以是空的，但它告诉 Python 这个文件夹是一个包。

```
my_project/

    main.py
```

```
mypackage/  
    __init__.py  
    utils.py  
    models.py  
    services.py
```

用了包之后，导入使用点号：

```
# main.py  
  
from mypackage import utils  
  
from mypackage.models import Student  
  
print(utils.add(1, 2))  
  
s = Student("张三", 80)
```

`__init__.py` 里可以写一些代码，在包导入时执行。也可以在里面定义 `__all__` 变量，控制 `from mypackage import *` 的行为。但更好的习惯是在 `__init__.py` 里集中导入常用的类和函数，简化外部调用者的导入路径：

```
# mypackage/__init__.py  
  
from .models import Student, Teacher  
  
from .utils import add, multiply
```

这样使用者只需要写：

```
from mypackage import Student, add
```

10.7 pip 与虚拟环境：安装别人的库，管理依赖

你已经用 `python -m pip install` 装过 `notebook` 了。`pip` 是 Python 的包管理器，从 PyPI（Python Package Index）下载安装包。

常用 `pip` 命令：

```
# 安装  
  
python -m pip install requests  
  
# 安装指定版本  
  
python -m pip install requests==2.28.0  
  
# 升级  
  
python -m pip install --upgrade requests  
  
# 卸载  
  
python -m pip uninstall requests  
  
# 列出已安装的包
```

```
python -m pip list

# 导出当前环境的所有包及其版本

python -m pip freeze > requirements.txt

# 从文件批量安装

python -m pip install -r requirements.txt
```

虚拟环境：隔离项目依赖

不同项目可能依赖不同版本的包。项目 A 用 requests 2.28，项目 B 用 requests 2.30，全局装一个会冲突。这时就该用虚拟环境——给每个项目独立的 Python 环境。

从 Python 3.3 开始，内置了 venv 模块：

```
# 在项目目录下创建虚拟环境

python -m venv myenv

# 激活

# Windows:

myenv\Scripts\activate

# Mac / Linux:

source myenv/bin/activate

# 激活后，命令行前会出现 (myenv) 提示符

# 现在 pip install 装的东西都在这个隔离环境里

# 退出虚拟环境

deactivate
```

新手坑位：虚拟环境不要提交到 git，不要放在项目文件夹里用同样的名字。

通常环境文件夹名叫 venv 或 .venv，并在 .gitignore 里加上它。这就是为什么项目中都有一个 requirements.txt——它记录了项目依赖，别人拿到代码一键安装即可，不需要拷贝虚拟环境。

10.8 标准库宝藏：Python 自带的强力工具箱

Python 自带了几百个标准库模块，是你免费获得的财富。下面列出一些你迟早会用到的，先混个脸熟：

模块	用途	示例
os	操作系统接口	os.listdir(".")列出当前目录文件
os.path	路径操作	os.path.join("dir", "file.txt")
pathlib	现代路径操作（3.4+）	Path("dir/file.txt").read_text()

模块	用途	示例
sys	系统相关参数	sys.argv 获取命令行参数
datetime	日期时间	datetime.datetime.now()
random	随机数	random.randint(1, 10)
json	JSON 处理	json.loads('{"a":1}')
csv	CSV 文件读写	csv.reader(f)
re	正则表达式	re.search(r"\d+", "abc123")
argparse	命令行参数解析	写 CLI 工具
math	数学函数	math.sqrt、math.pi
statistics	统计函数	statistics.mean([1,2,3])
itertools	迭代器工具	itertools.permutations
collections	高级数据结构	Counter、defaultdict

这里重点展示几个你会高频使用的。

os 和 pathlib: 处理文件路径

传统方式用 os.path:

```
import os
print(os.getcwd())           # 当前工作目录
print(os.path.exists("test.txt")) # 检查文件是否存在
print(os.path.join("dir", "subdir")) # 跨平台的路径拼接
```

更现代的方式用 pathlib (推荐):

```
from pathlib import Path
p = Path("data/novel.txt")
print(p.exists())           # 是否存在
print(p.read_text())        # 直接读成字符串
print(p.parent)             # 上级目录 Path("data")
# 遍历目录下所有 txt 文件
```

```
for f in Path(".").glob("*.txt"):
    print(f)
```

json: 结构化数据交换

```
import json
data = {"name": "张三", "age": 25, "scores": [88, 92]}
# Python 对象 → JSON 字符串
json_str = json.dumps(data, ensure_ascii=False, indent=2)
print(json_str)
# JSON 字符串 → Python 对象
parsed = json.loads(json_str)
print(parsed["name"])
```

ensure_ascii=False 让中文正常显示, indent=2 让输出格式化。

random: 随机数

```
import random
print(random.randint(1, 10))      # 1-10 的随机整数
print(random.choice(["a", "b", "c"])) # 随机选一个
print(random.sample(range(100), 5)) # 随机抽 5 个不重复
```

requests: 发送网络请求 (第三方库, 但几乎人手必备)

需要先安装: `python -m pip install requests`

```
import requests
response = requests.get("https://api.github.com")
print(response.status_code) # 200
data = response.json()
print(data.keys())
collections.Counter: 用专业的做词频统计
```

还记得第 7 章我们手写了几十行代码做词频统计吗? 用标准库直接搞定:

```
from collections import Counter
words = ["a", "b", "c", "a", "b", "a"]
counter = Counter(words)
print(counter)                # Counter({'a': 3, 'b': 2, 'c': 1})
print(counter.most_common(2)) # [('a', 3), ('b', 2)]
```

一行抵我们好几行代码。知道标准库有什么，能帮你省下大量时间。

10.9 实战：用 requests 写个命令行天气查询工具

我们写一个能调用的工具脚本：输入城市名，输出天气信息。这里用免费的天气 API (<https://wttr.in>) ——不需要 API key，直接在 URL 里指定城市即可获得格式化天气。

```
# weather.py

"""
命令行天气查询工具。

用法: python weather.py 北京
"""

import sys
import requests

def get_weather(city):
    """返回指定城市的天气信息"""
    url = f"https://wttr.in/{city}?format=%l:+%c+%t&lang=zh"
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        return response.text.strip()
    except requests.exceptions.RequestException as e:
        return f"获取天气失败: {e}"

def main():
    if len(sys.argv) < 2:
        print("用法: python weather.py <城市名>")
        print("示例: python weather.py 北京")
        sys.exit(1)

    city = sys.argv[1]
    weather = get_weather(city)
    print(weather)

if __name__ == "__main__":
    main()
```

运行 `python weather.py 北京`，输出类似北京: +20° C。

工作原理：

`sys.argv` 是命令行参数列表，`sys.argv[0]` 是脚本名，`sys.argv[1]` 是你传的第一个参数。

`requests.get()` 发 HTTP 请求，`raise_for_status()` 在 HTTP 状态码非 200 时抛异常。

主程序包装在 `main()` 函数里，然后用 `if __name__ == "__main__"` 启动。这是 CLI 脚本的标准模式。如果你的脚本既可被导入又可独立运行，这种结构是铁律。

10.10 总结与下一步

这一章，你从写单个文件跨越到了组织项目和利用外部世界的力量：

模块：`.py` 文件即模块，`import` 引入。

包：含 `__init__.py` 的文件夹，用点号组织命名空间。

`if __name__ == "__main__"`：区分导入和直接运行，每个脚本的标准收尾。

`pip`：安装第三方库，`pip freeze` 导出依赖。

虚拟环境：隔离项目依赖，`python -m venv` 创建，`activate` 激活。

标准库：免费的宝藏在等你探索，`collections.Counter` 一行顶几十行。

实战工具：命令行天气查询，用到了 `sys.argv`、`requests`、异常处理。

现在，你已经能安装库、导入库、自己写模块、把代码组织成包。下一章起，我们将进入高阶编程的核心区域。首先要讲的是高阶函数、迭代器、生成器与装饰器——它们是 Python “魔法” 的来源，也是最让人觉得 “优雅” 的地方。

练习

1. 将第 7 章词频统计的功能改写为一个模块 `wordcounter.py`，包含一个 `count_words(filename)` 函数，返回词频字典；在 `if __name__ == "__main__"` 块里写一个测试：对一个测试文件统计词频并打印前 10 个。

2. 写一个模块 `math_utils.py`，包含 `circle_area(radius)` 和 `sphere_volume(radius)` 两个函数（球体积公式 $V = 4/3 \pi r^3$ ）。在另一个脚本里导入并使用它们。

3. 创建一个包 `mylib`，里面包含 `__init__.py`（导入 `calculator` 的 `add` 函数）、`calculator.py`（定义 `add` 和 `subtract` 函数）、`greetings.py`（定义 `hello(name)`）。写一个 `main.py` 导入并使用。

4. 用 `os` 或 `pathlib` 遍历当前目录，列出所有 `.py` 文件及其大小。

5. 创建一个虚拟环境，激活它，安装 `requests` 库，用 `pip freeze` 导出 `requirements.txt`。

练习参考答案

练习 1

```
# wordcounter.py

import string

def count_words(filename):
```

```

"""统计文件中单词出现次数，返回字典"""

word_counts = {}

try:
    with open(filename, "r", encoding="utf-8") as f:
        for line in f:
            line = line.lower()
            line = line.translate(str.maketrans("", "", string.punctuation))
            for word in line.split():
                word_counts[word] = word_counts.get(word, 0) + 1
except FileNotFoundError:
    print(f"文件 {filename} 不存在")
    return {}

return word_counts

if __name__ == "__main__":
    # 测试

    counts = count_words("test.txt") # 需要自己创建一个 test.txt
    top10 = sorted(counts.items(), key=lambda x: x[1], reverse=True)[:10]
    for word, freq in top10:
        print(f"{word}: {freq}")

```

练习 2

```

# math_utils.py

import math

def circle_area(radius):
    return math.pi * radius ** 2

def sphere_volume(radius):
    return (4/3) * math.pi * radius ** 3

if __name__ == "__main__":
    print(circle_area(5))
    print(sphere_volume(5))

# main.py

from math_utils import circle_area, sphere_volume

```

```
print(f"半径 5 的圆面积: {circle_area(5):.2f}")
print(f"半径 5 的球体积: {sphere_volume(5):.2f}")
```

练习 3

目录结构:

```
mylib/
    __init__.py
    calculator.py
greetings.py
main.py
# mylib/calculator.py
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
# mylib/greetings.py
def hello(name):
    return f"你好, {name}! "
# mylib/__init__.py
from .calculator import add
# main.py
from mylib import add
from mylib.greetings import hello
print(add(10, 20))
print(hello("张三"))
```

练习 4

```
from pathlib import Path
for f in Path(".").glob("*.py"):
    size = f.stat().st_size
    print(f"{f.name}: {size} 字节")
```

练习 5

```
# 创建虚拟环境
```

```
python -m venv venv  
  
# 激活 (Windows)  
venv\Scripts\activate  
  
# 激活 (Mac/Linux)  
source venv/bin/activate  
  
# 安装  
python -m pip install requests  
  
# 导出  
python -m pip freeze > requirements.txt
```

第 11 章 高阶函数与迭代器魔术：让代码更 Pythonic

你已经知道怎么定义函数、调用函数。但函数本身在 Python 里只是一种更聪明的“对象”——它可以被赋值给变量，可以当作参数传给另一个函数，甚至可以在函数里定义函数。这种能把函数像数据一样摆弄的能力，带来了非常优雅的表达方式。

同时，你处理过大量数据，可能遇到过“一次性把所有数据塞进内存导致卡死”的情况。Python 提供了生成器，让你可以像流水线一样处理数据，即使文件有几百 GB，内存也只占一点点。

最后，我们会学到装饰器——它可以给函数“穿衣服”，在不改动原函数代码的前提下增加

新行为。这听起来有点玄，但它正是 Python 里非常常见的“魔法”来源。

这一章，我们将揭开这些高级特性的面纱。学完之后，你不仅能写出功能正确的代码，还能写出简洁、高效、有“Python 味”的代码。

11.1 函数是一等公民：可以赋值、传参、返回

Python 中一切皆对象。函数也不例外。你定义一个函数，实际上就是创建了一个 function 类型的对象，把函数名作为标签贴上去。

```
def greet(name):  
    return f"你好, {name}!"  
  
# 函数可以赋值给另一个变量  
  
hello = greet  
  
print(hello("张三")) # 你好, 张三!
```

看到了吗？hello 现在也指向同一个函数对象，你可以通过它来调用。

因为函数是对象，它也能作为参数传递给另一个函数。这就是高阶函数的标志——接受函数作为参数，或者返回一个函数。

```
def apply(func, value):  
    return func(value)  
  
print(apply(greet, "李四")) # 你好, 李四!  
  
print(apply(len, "Python")) # 6
```

apply 接受任何单参数函数，然后对 value 调用它。

函数还可以在内部定义函数并返回：

```
def make_multiplier(factor):  
    def multiply(x):  
        return x * factor  
    return multiply  
  
double = make_multiplier(2)  
triple = make_multiplier(3)  
  
print(double(5)) # 10  
print(triple(5)) # 15
```

make_multiplier 就像一个“函数工厂”，根据你给的 factor 生产出不同倍数的乘法函数。内部函数 multiply 能够记住它出生时外部的 factor 变量——这就是闭包。我们后面在装饰器里会再次用到。

11.2 经典高阶函数：map、filter、reduce

Python 内置了三个著名的高阶函数，它们源自函数式编程，但这些年用得越来越谨慎。知道它们能读懂别人的代码，但自己写时不要滥用。

map：对每个元素应用同一个函数

```
nums = [1, 2, 3, 4]
squared = map(lambda x: x**2, nums)
print(list(squared)) # [1, 4, 9, 16]
```

map 返回一个迭代器（下一节细说），需要用 list() 转换成列表。其实有更清晰的写法——列表推导式：

```
squared = [x**2 for x in nums] # 一样的效果，更 Pythonic
```

大部分时候，列表推导式比 map 更好读。

filter：按条件筛选元素

```
nums = [10, 15, 20, 25, 30]
even = filter(lambda x: x % 2 == 0, nums)
print(list(even)) # [10, 20, 30]
```

同样，列表推导式能做得更漂亮：

```
even = [x for x in nums if x % 2 == 0]
```

reduce：累积运算

reduce 从序列中依次取出元素，和一个累积值做运算，最终得出一个单一结果。例如累加：

```
from functools import reduce
nums = [1, 2, 3, 4]
total = reduce(lambda acc, x: acc + x, nums, 0)
print(total) # 10
```

不过，累加直接 sum(nums) 就行，完全没必要 reduce。只有在没有直接工具可用时才考虑 reduce，比如连乘：

```
product = reduce(lambda acc, x: acc * x, nums, 1)
```

原则：在可读性面前，函数式不是信仰。能用列表推导、生成器表达式、内置函数（sum、any、all）解决的问题，就不要硬上 map/filter/reduce。

11.3 生成器：懒加载的流水线

你在第 7 章读取大文件时，用 for line in f: 逐行读取，内存占用极小。这就是一种生成器思维——数据按需产生，而不是一次性全拉出来。

yield：让函数变成生成器

一个包含 yield 语句的函数，调用时不会执行函数体，而是返回一个生成器对象。每次对它 next() 或迭代，它就跑到下一个 yield 处暂停，把值交出来。

```
def countdown(n):  
    while n > 0:  
        yield n  
        n -= 1  
  
for num in countdown(5):  
    print(num)    # 5 4 3 2 1
```

yield 就像函数里的一个“检查点”，每次访问都从上次暂停处继续，直到没有 yield 了，自动结束。

生成器最大的意义：它不一次性产生所有值，而是产生一个算一个。如果你要生成一亿个数字，用列表会撑爆内存：

```
# 会爆内存  
big_list = [i for i in range(10**8)]  
  
# 完全没事，几乎不占内存  
big_gen = (i for i in range(10**8))
```

(i for i in range(...)) 这种圆括号的写法是生成器表达式，和列表推导式长得一模一样，只是把方括号换成圆括号。它是创建生成器最轻松的方式。

生成器链：像流水线一样处理数据

你可以把多个生成器串在一起，形成一条数据处理管道，每个环节只处理当前这个数据项，整个流程内存极省。

```
# 读取大文件 → 过滤空行 → 转小写 → 取出包含 python 的行  
with open("big_log.txt") as f:  
    lines = (line.strip() for line in f)    # 去空白  
    non_empty = (line for line in lines if line) # 过滤空行  
    lower_lines = (line.lower() for line in non_empty)  
    python_lines = (line for line in lower_lines if "python" in line)  
    for line in python_lines:  
        print(line)
```

每一步返回一个生成器，数据像水一样流过管道，绝不积压。

11.4 装饰器：给函数“穿衣服”

想象你写了很多函数，现在想给每个函数添加上“开始/结束日志”，或者“计算执行时间”。如果逐个去改函数内部代码，既麻烦又容易漏。装饰器允许你在不修改函数源码的情况下，给它

包裹一层新行为。

无参装饰器的基本形态

装饰器本质上是一个接受函数并返回新函数的高阶函数。

```
def logger(func):  
    def wrapper(*args, **kwargs):  
        print(f"调用 {func.__name__}, 参数: {args}, {kwargs}")  
        result = func(*args, **kwargs)  
        print(f"{func.__name__} 返回 {result}")  
        return result  
    return wrapper
```

这里 wrapper 使用了 *args, **kwargs 来接受任意参数，确保任何函数都能被装饰。内部调用原函数 func，并拿到结果，前后加日志。

使用装饰器语法@:

```
@logger  
def add(a, b):  
    return a + b  
print(add(3, 5))
```

输出:

```
#调用 add, 参数: (3, 5), {}  
#add 返回 8  
8
```

@logger 就是 add = logger(add)的语法糖——把 add 传进 logger, 返回的 wrapper 再赋值给 add。从此调用 add 其实是在调用 wrapper。

新手坑位：原函数的元信息（函数名、文档字符串）被 wrapper 覆盖了。

上面的 add.__name__ 变成了 "wrapper"。要修复，用标准库的 functools.wraps:

```
from functools import wraps  
def logger(func):  
    @wraps(func)  
    def wrapper(*args, **kwargs):  
        print(f"调用 {func.__name__}")  
        return func(*args, **kwargs)  
    return wrapper
```


@wraps(func)把原函数的__name__、__doc__等属性复制到 wrapper 上。以后写装饰器，请永远加上@wraps。

带参数的装饰器

有时你想让装饰器接受额外的参数，比如给日志指定级别。这就需要再包一层——装饰器工厂。

```
def log_with_level(level):  
    def decorator(func):  
        @wraps(func)  
        def wrapper(*args, **kwargs):  
            print(f"[{level}] 调用 {func.__name__}")  
            return func(*args, **kwargs)  
        return wrapper  
    return decorator  
  
@log_with_level("DEBUG")  
def multiply(a, b):  
    return a * b  
  
multiply(4, 5) # [DEBUG] 调用 multiply
```

执行流程：log_with_level("DEBUG")先执行，返回真正的装饰器 decorator，再由@应用到 multiply。三层嵌套刚接触可能有点晕，但记住模板即可：外层接收装饰器参数，中层接收函数，内层是 wrapper。

11.5 实战：计时装饰器 + 惰性数据处理

我们写一个能计算函数运行时间的装饰器，并用在数据处理函数上。

```
import time  
  
from functools import wraps  
  
def timer(func):  
    """打印函数运行时间"""  
    @wraps(func)  
    def wrapper(*args, **kwargs):  
        start = time.perf_counter()  
        result = func(*args, **kwargs)  
        elapsed = time.perf_counter() - start  
        print(f"{func.__name__} 运行时间：{elapsed:.4f} 秒")  
    return wrapper
```

```

        return result

    return wrapper

@timer
def process_data(filename):
    # 模拟一个耗时操作：读取文件，统计行数，并返回生成器

    def line_generator():
        with open(filename, "r", encoding="utf-8") as f:
            for line in f:
                # 假设每行有些处理

                yield line.strip()

    # 生成器本身不耗时，真正耗时的是后续遍历

    lines = line_generator()

    # 强行遍历以模拟耗时

    count = sum(1 for _ in lines)

    return count

# 自己先创建一个测试文件 test.txt 写入几行文字

print(process_data("test.txt"))

```

注意：生成器的定义和返回几乎不耗时，真正的处理在迭代时才发生。我们这里用 `sum(1 for _ in lines)` 强制遍历统计，触发了实际处理。

11.6 总结与下一步

这一章，你触碰了 Python 里一些优雅的高级特性：

函数是一等公民：可以赋值、传参、返回。

高阶函数：map/filter/reduce，但要克制，优先使用可读性更强的列表推导和内置函数。

生成器：yield 和生成器表达式，按需产生数据，节省内存，建立数据处理管道。

装饰器：给函数包裹新行为，修改原函数零侵入。务必使用 @wraps 保留元信息。

这些特性不是必须的，但它们能让你写出更简洁、更高效的代码。越往后的项目，你会越依赖装饰器（如 Flask 的路由 `@app.route("/")`）和生成器（如处理大数据）。

下一章，我们将进入数据科学领域的前哨战——NumPy 与 Pandas 入门。你会发现，那些让原生 Python 跑得喘粗气的数组计算，它们秒秒钟搞定。

练习

1. 编写一个接受函数和列表的高阶函数 `apply_to_all(func, lst)`，返回一个新列表，元素是 `func` 对每个元素调用的结果。分别用 `map` 和自己手写循环实现一次。

2. 创建一个生成器 `fibonacci(n)`，生成前 `n` 个斐波那契数。用 `for` 循环测试。
3. 写一个装饰器 `double_result`，让被装饰函数的返回值翻倍（只对数值有效，如果是其他类型原样返回）。
4. 使用生成器表达式，对一个大文件逐行读取，过滤掉空行，将每行转为大写，然后打印前 5 行（用 `itertools.islice` 或手动计数）。
5. 实现一个带参数的装饰器 `retry(times)`，如果被装饰函数抛出异常，就重试最多 `times` 次，每次间隔 0.5 秒。如果全部失败，打印错误并重新抛出最后的异常。

练习参考答案

练习 1

```
def apply_to_all(func, lst):  
    result = []  
    for item in lst:  
        result.append(func(item))  
    return result  
  
# 或直接用 map  
def apply_to_all_map(func, lst):  
    return list(map(func, lst))  
  
print(apply_to_all(lambda x: x**2, [1,2,3])) # [1,4,9]
```

练习 2

```
def fibonacci(n):  
    a, b = 0, 1  
    for _ in range(n):  
        yield a  
        a, b = b, a + b  
  
for num in fibonacci(10):  
    print(num) # 0 1 1 2 3 5 8 13 21 34
```

练习 3

```
from functools import wraps  
  
def double_result(func):  
    @wraps(func)  
    def wrapper(*args, **kwargs):  
        result = func(*args, **kwargs)
```

```

        if isinstance(result, (int, float)):
            return result * 2

        return result

    return wrapper

@double_result
def add(a, b):
    return a + b

@double_result
def greet(name):
    return f"Hello {name}"

print(add(3, 4))      # 14
print(greet("Bob"))   # Hello Bob

```

练习 4

```

# 假设 big.txt 存在, 多行文本
with open("big.txt", "r") as f:
    upper_lines = (line.upper() for line in f if line.strip())
    for i, line in enumerate(upper_lines):
        if i >= 5:
            break
        print(line, end="")

```

练习 5

```

import time

from functools import wraps

def retry(times):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            last_exception = None
            for attempt in range(times):
                try:

```

```
        return func(*args, **kwargs)

    except Exception as e:

        last_exception = e

        print(f"重试 {attempt+1}/{times}, 因为 {e}")

        time.sleep(0.5)

    print(f"全部 {times} 次重试失败")

    raise last_exception

    return wrapper

return decorator

@retry(3)
def unstable_func():

    # 模拟失败

    import random

    if random.random() < 0.7:

        raise ValueError("连接失败")

    return "成功"

# 试试看

print(unstable_func())
```

第 12 章 高效数据处理：NumPy 与 Pandas 入门

你已经能用 Python 的列表、字典、循环处理数据了。但当你面对一个 10 万行的 CSV 表格，想算一列的平均值，用原生列表加循环会又慢又繁琐。更别提真正的数据科学、机器学习领域，动辄上百万条数据——那种场景下，原生 Python 循环就像用勺子挖运河。

Python 在数据科学领域的统治地位，靠的是两大神器：NumPy（高性能数组计算）和 Pandas（表格数据处理）。它们底层用 C 语言实现，把循环推到了 C 层执行，速度提升成百上千倍。更重要的是，它们提供了非常直观的接口，让复杂的数据操作变成一两行代码。

这一章，我们从零开始接触这两把利器。学完之后，处理表格数据、做统计分析、画简单图表，你会感觉像呼吸一样自然。

12.1 为什么原生列表不够用？

看一个场景：你有一个列表，存了 100 万个数字，想算它们的平方和。原生 Python：

```
import time

data = list(range(1_000_000))

start = time.time()

result = sum(x**2 for x in data)

print(f"耗时：{time.time() - start:.3f} 秒")
```

在我的电脑上大约 0.1 秒，这还行。但如果数据再大点，或者在循环中嵌套循环，速度会急剧下降。原生 Python 慢在：每次 `x**2` 都在 Python 层面做了类型检查、方法分派等大量额外工作。

NumPy 的做法：把数据存进一个数组，所有元素类型统一（比如全是 64 位整数）。然后 `arr**2` 这条指令直接在 C 层面遍历、计算，没有 Python 层的额外开销。同样 100 万个数字：

```
import numpy as np

arr = np.arange(1_000_000)

start = time.time()

result = np.sum(arr ** 2)

print(f"耗时：{time.time() - start:.4f} 秒")
```

耗时会降到几毫秒。把循环推到底层去执行，这就是 NumPy 的核心价值。

12.2 NumPy 数组：比列表快百倍的同质数据容器

安装：

```
python -m pip install numpy
```

创建数组：

```
import numpy as np    # 行业惯用别名

# 从列表创建
```

```

a = np.array([1, 2, 3, 4])
print(a)          # [1 2 3 4]
print(type(a))    # <class 'numpy.ndarray'>

# 快速创建序列

print(np.arange(0, 10, 2))    # [0 2 4 6 8] 类似 range
print(np.linspace(0, 1, 5))  # [0.  0.25 0.5 0.75 1.] 均匀分布

# 创建全零、全一阵列

print(np.zeros((3, 4)))      # 3 行 4 列全 0
print(np.ones((2, 3)))       # 2 行 3 列全 1
print(np.eye(3))             # 3x3 单位矩阵

```

数组属性:

```

arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr.shape)    # (2, 3) 形状: 2 行 3 列
print(arr.ndim)     # 2      维度数
print(arr.size)     # 6      元素总数
print(arr.dtype)    # int32 或 int64, 取决于系统和输入

```

向量化运算: 告别循环:

```

a = np.array([1, 2, 3, 4])
b = np.array([5, 6, 7, 8])

print(a + b)        # [ 6  8 10 12]
print(a * b)        # [ 5 12 21 32] 逐元素乘
print(a ** 2)       # [ 1  4  9 16]
print(np.sqrt(a))   # [1.  1.414 1.732 2.]
print(a > 2)        # [False False  True  True]

```

新手坑位: *是逐元素乘, 不是矩阵乘法。

矩阵乘法用@或 np.dot(a, b)。

索引与切片

和列表类似, 但支持多维切片:

```

arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(arr[1, 2])    # 6    第 2 行第 3 列

```

```
print(arr[:, 1])      # [2 5 8]   所有行的第 2 列
print(arr[:2, 1:])    # [[2 3] [5 6]] 前 2 行, 从第 2 列开始
```

布尔索引：按条件筛选

```
scores = np.array([88, 92, 67, 74, 95])
mask = scores >= 80
print(mask)          # [ True  True False False  True]
print(scores[mask])   # [88 92 95]
print(scores[scores >= 80]) # 组合写法
```

统计函数

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(np.mean(arr))      # 3.5   总平均值
print(np.mean(arr, axis=0)) # [2.5 3.5 4.5] 每列平均值
print(np.mean(arr, axis=1)) # [2. 5.]  每行平均值
print(np.sum(arr))       # 21
print(np.max(arr))       # 6
print(np.argmax(arr))    # 5   最大值的索引（展平后）
```

axis=0 是沿行的方向压缩（对列操作），axis=1 是沿列的方向压缩（对行操作）。初学可能混淆，画个图或试两遍就熟了。

12.3 Pandas：表格数据的瑞士军刀

如果 NumPy 是发动机，Pandas 就是整辆车。它提供了两个核心数据结构：

Series：带索引的一维数组（就是有名字标签的一系列数据）

DataFrame：二维表格，由多个 Series 拼成，像 Excel 工作表

安装

```
python -m pip install pandas
```

Series 基础

```
import pandas as pd
s = pd.Series([88, 92, 67, 74, 95], index=["张三", "李四", "王五", "赵六", "钱七"])
print(s)
```

输出：

张三	88
李四	92


```
钱五    67
赵六    74
宋七    95

dtype: int64
```

通过索引取值：

```
print(s["张三"])      # 88
print(s) # 取多个
```

DataFrame 基础

DataFrame 是最常用的结构。创建方式很多，最常见的是从字典创建：

```
data = {
    "姓名": ["张三", "李四", "王五", "赵六"],
    "年龄": [18, 19, 18, 20],
    "分数": [88, 92, 67, 74]
}
df = pd.DataFrame(data)
print(df)
```

输出：

```
   姓名  年龄  分数
0  张三   18   88
1  李四   19   92
2  王五   18   67
3  赵六   20   74
```

常见的查看方法：

```
print(df.head(2))      # 前 2 行
print(df.tail(1))      # 最后 1 行
print(df.shape)        # (4, 3)
print(df.columns)      # Index(['姓名', '年龄', '分数'])
print(df.info())       # 每列的数据类型和缺失情况
print(df.describe())   # 数值列的统计摘要（均值、标准差、四分位数等）
```

选择数据

```
# 选一列：返回 Series
```

```

print(df["分数"])

print(df.分数)          # 也可以用属性式写法，但列名有空格或与内置名冲突时不行

# 选多列：返回 DataFrame

print(df)

# 选行：用 .loc[行标签] 和 .iloc[行位置]

print(df.loc[1])        # 按索引标签（默认索引就是行号）

print(df.iloc[0])       # 按位置，第一行

print(df.iloc[1:3])     # 第 2 到第 3 行（不含第 4 行）

# 行列同时选

print(df.loc[1, "分数"]) # 第 2 行分数列

print(df.iloc[0:2, [0, 2]]) # 前 2 行，第 1 列和第 3 列

```

新手坑位：.loc 包含结束索引，.iloc 不包含。

df.loc[0:2]返回 0、1、2 三行；df.iloc[0:2]返回 0、1 两行。这是 Pandas 存在的一个设计不对称点。

按条件过滤

```

# 分数大于 80 的所有行

high = df[df["分数"] > 80]

print(high)

# 多重条件：分数>75 且 年龄<20

condition = (df["分数"] > 75) & (df["年龄"] < 20)

print(df[condition])

```

注意：这里必须用&（and）和（or），不能用 and/or 关键字。每个条件必须加括号。

处理缺失值

现实数据总有缺失。Pandas 用 NaN 表示缺失值：

```

import numpy as np

df2 = pd.DataFrame({
    "A": [1, 2, np.nan, 4],
    "B": [np.nan, 5, 6, np.nan]
})

print(df2.isnull())      # 每个格是否为缺失

print(df2.dropna())      # 删除含缺失值的行

```

```
print(df2.fillna(0))      # 用 0 填充缺失值

print(df2["A"].fillna(df2["A"].mean())) # 用平均值填充
```

分组聚合：类似 SQL 的 GROUP BY

```
# 按年龄分组，计算各组的平均分数

print(df.groupby("年龄")["分数"].mean())
```

输出：

```
年龄
18    77.5
19    92.0
20    74.0
```

可以同时算多个统计量：

```
print(df.groupby("年龄").agg({"分数": ["mean", "max", "count"]}))
```

读写文件

Pandas 支持直接读写 CSV、Excel、JSON 等：

```
# 读 CSV

df = pd.read_csv("data.csv", encoding="utf-8")

# 写 CSV

df.to_csv("output.csv", index=False, encoding="utf-8") # index=False 不写入行号

# 读 Excel（需要 pip install openpyxl）

df = pd.read_excel("data.xlsx")
```

12.4 实战：清洗并分析一份脏数据

假设你拿到一份学生成绩数据 `scores.csv`，但很脏：有缺失值、有重复记录、有异常分数。要求：清洗数据，计算每人的总分和平均分，标出是否全科及格，最后输出一个干净的汇总表。

原始 CSV 内容示例：

```
姓名,语文,数学,英语
张三,85,90,78
李四,92,,88
王五,67,120,74
赵六,74,82,
张三,85,90,78
```

模拟实战代码：

```
import pandas as pd

import numpy as np

# 1. 读取

df = pd.read_csv("scores.csv")

# 2. 初步查看

print("原始数据: ")

print(df.info())

print(df.describe())

# 3. 去重

df = df.drop_duplicates()

print(f"去重后行数: {len(df)}")

# 4. 处理缺失值 — 用各科中位数填充

for col in ["语文", "数学", "英语"]:

    median = df[col].median()

    df[col].fillna(median, inplace=True) # inplace=True 在原 DataFrame 上改

# 5. 处理异常分数 — 将数学超过 100 的更正为 NaN, 再统一填充

df.loc[df["数学"] > 100, "数学"] = np.nan

df["数学"].fillna(df["数学"].median(), inplace=True)

print("\n清洗后数据: ")

print(df)


# 6. 计算总分和平均分

df["总分"] = df["语文"] + df["数学"] + df["英语"]

df["平均分"] = df["总分"] / 3

# 7. 判断是否全科及格 (每科  $\geq 60$ )

df["全科及格"] = (df["语文"] >= 60) & (df["数学"] >= 60) & (df["英语"] >= 60)

# 8. 排序输出

df = df.sort_values("总分", ascending=False)

print("\n汇总: ")

print(df)

# 9. 保存
```

```
df.to_csv("cleaned_scores.csv", index=False, encoding="utf-8-sig") #  
utf-8-sig 避免 Excel 乱码
```

代码解读：

drop_duplicates()删除完全重复的行。

fillna()填充缺失值，整体中位数比均值更不易受极端值影响。

df.loc[条件, 列名]定位并修改特定单元格。

生成新列只需 df["新列名"]= 表达式，这就是“派生列”。

utf-8-sig 编码在 Windows Excel 下不易乱码。

12.5 快速可视化：matplotlib 一句话画图

数据分析后往往要一张图。matplotlib 是 Python 最老牌的画图库，Pandas 直接内置了简易接口。

安装：

```
python -m pip install matplotlib
```

代码：

```
import matplotlib.pyplot as plt  
# 不离开 DataFrame 直接画  
df.plot(kind="bar", x="姓名", y="总分", title="学生总成绩")  
plt.tight_layout()  
plt.show()
```

常用 kind: "line" (折线)、"bar" (柱)、"barh" (横向柱)、"hist" (直方图)、"scatter" (散点图)、"box" (箱线图)。

12.6 总结与下一步

这一章，你从纯 Python 数据处理跨入了高性能计算和分析的世界：

NumPy：用 ndarray 做向量化运算，速度百倍提升，是整个科学计算生态的根基。

Pandas：以 Series 和 DataFrame 为中心，提供表格数据的读取、清洗、过滤、分组、聚合等全套工具。

实战：走了完整的数据清洗+分析流程，最终输出干净的结果和图表。

你现在的工具箱已经相当丰富。但目前为止，所有的程序都是“单线程”的——一次只做一件事。如果同时下载 100 个网页，一个一个排队等待，时间累加起来会很可怕。下一章，我们将学习网络与并发：多线程、多进程、异步 IO。让你的程序学会“同时做很多事”，大幅提升效率。

练习

- 1.用 NumPy 创建一个 3x3 的随机数组 (np.random.rand()), 计算每行和每列的总和。
- 2.用 Pandas 读取本章的 scores.csv 示例数据,计算每人的语数英三科“标准差”(df.std(axis=1)), 并添加到新列“标准差”。
- 3.用 Pandas 的 groupby 和 agg, 对 df 按年龄分组, 分别统计分数的平均值、最大值、人数。
- 4.模拟一个 100 万条数据的 DataFrame (用 np.random.randn 生成两列随机数), 分别用原生 Python 循环和 NumPy/Pandas 计算两列乘积之和, 感受速度差异。
- 5.用 matplotlib 画一个简单的折线图, 展示某股票一周的收盘价走势 (自己编数据即可)。

练习参考答案

练习 1

```
import numpy as np
arr = np.random.rand(3, 3)
print("数组: \n", arr)
print("行和: ", arr.sum(axis=1))
print("列和: ", arr.sum(axis=0))
```

练习 2

```
import pandas as pd
df = pd.read_csv("scores.csv")
# 先处理可能的缺失值
df.fillna(0, inplace=True)
df["标准差"] = df.std(axis=1)
print(df)
```

练习 3

```
result = df.groupby("年龄").agg(
    平均分=("分数", "mean"),
    最高分=("分数", "max"),
    人数=("分数", "count")
)
print(result)
```

练习 4

```
import numpy as np
```

```

import pandas as pd

import time

# 模拟数据

n = 1_000_000

np.random.seed(42)

a = np.random.randn(n)

b = np.random.randn(n)


# 原生 Python 循环

start = time.time()

sum_py = sum(a[i] * b[i] for i in range(n))

print(f"Python 循环耗时: {time.time() - start:.3f} 秒")

# NumPy 向量化

start = time.time()

sum_np = np.sum(a * b)

print(f"NumPy 向量化耗时: {time.time() - start:.4f} 秒")

# Pandas

df = pd.DataFrame({"a": a, "b": b})

start = time.time()

sum_pd = (df["a"] * df["b"]).sum()

print(f"Pandas 耗时: {time.time() - start:.4f} 秒")

```

速度差异通常在几十到几百倍。

练习 5

```

import matplotlib.pyplot as plt

days = ["周一", "周二", "周三", "周四", "周五"]

prices = [28.5, 29.0, 30.2, 29.8, 31.5]

plt.plot(days, prices, marker="o", linestyle="--", color="blue")

plt.title("某股票一周收盘价")

plt.xlabel("日期")

plt.ylabel("价格 (元)")

plt.grid(True)

```

```
plt.show()
```

第 13 章 网络与并发：让程序更快更强大

到目前为止，你写的程序都在自己的电脑里运行，数据来自键盘、文件和 CSV。但现代程序的大部分力量来自网络——从网页抓取信息、调用 API、读写云端数据库。一个真正的 Python 开发者，迟早要让代码跑在互联网上。

同时，程序经常需要“同时做很多事”。比如你要下载 100 个网页。如果按顺序一个一个下载，每个等 0.1 秒，总共就是 10 秒。但如果能让 100 个下载几乎同时进行，耗时可能不到 1 秒。这就是并发要解决的问题。

这一章，我们从网络基础开始，然后进入并发世界，认识线程、进程和异步 IO。最后我们会写一个礼貌的异步爬虫。学完之后，你的程序能从“单机玩具”变身“网络工具”。

13.1 网络编程基础：从套接字到 HTTP

网络通信的底层模型是套接字（socket）。你可以把它想成电话：一台电脑拨号，另一台接听，都拿着听筒，两端就建立了一条可以双向通话的通道。

Python 提供了 socket 模块做底层网络编程，但我们大多数时候不需要直接操作套接字。我们上网访问网页，基于 HTTP 协议——一种建立在套接字之上的请求/响应协议。

13.1.1 一个极简 HTTP 请求

你在浏览器输入 `http://example.com` 然后回车，背后的流程是：

1. 浏览器解析网址，提取域名和端口（默认 80）
2. 通过 DNS 将域名变成 IP 地址
3. 建立到该 IP 的 TCP 连接
4. 发送一个 HTTP 请求文本
5. 读取服务器返回的响应文本
6. 浏览器渲染响应内容（HTML）

用 Python 的 socket 可以手动完成这个过程。不过我们绝大多数时候用 requests 库，它封装了所有细节。但知道底层能帮助你理解网络问题的本质。

13.2 requests 库深度使用

第 10 章我们简要用过 requests，这里补充几个关键概念和常见操作。

发送 GET 请求并处理响应

```
import requests

response = requests.get("https://api.github.com")

print(response.status_code)      # 200
```



```
print(response.headers["Content-Type"]) # application/json; charset=utf-8
data = response.json()                 # 如果内容是 JSON，直接解析成字典
print(data.keys())
```

携带参数、请求头

URL 查询参数

```
params = {"q": "python", "sort": "stars"}
response = requests.get("https://api.github.com/search/repositories",
                        params=params)
# 自定义请求头，模拟浏览器
headers = {"User-Agent": "Mozilla/5.0"}
response = requests.get("https://www.baidu.com", headers=headers)
```

POST 请求发送数据

```
data = {"username": "test", "password": "123456"}
response = requests.post("https://httpbin.org/post", data=data)
print(response.json()) # httpbin 回显你发送的数据
```

超时与异常处理

新手坑位：永远给网络请求加 timeout。

如果对方服务器卡住或者网络断了，没有 timeout 的请求会永久挂起，程序卡死。

```
try:
    response = requests.get("https://example.com", timeout=10) # 10 秒超时
    response.raise_for_status() # 状态码不是 200 时抛出异常
except requests.exceptions.Timeout:
    print("请求超时")
except requests.exceptions.ConnectionError:
    print("连接错误，检查网络或域名")
except requests.exceptions.HTTPError as e:
    print(f"HTTP 错误: {e}")
except requests.exceptions.RequestException as e:
    print(f"请求异常: {e}")
```

raise_for_status()是一个非常实用的方法，省去我们手动检查状态码。

处理大文件下载（流式下载）

如果下载一个大文件，用 response.content 会一次性把整个文件加载到内存，内存会爆。用

流式下载：

```
with requests.get("https://example.com/bigfile.zip", stream=True) as r:
    r.raise_for_status()

    with open("bigfile.zip", "wb") as f:
        for chunk in r.iter_content(chunk_size=8192): # 每次读 8KB
            f.write(chunk)
```

逐块读取写入，内存占用极小。

13.3 并发难题：一个下载很慢，一万个呢？

现在你有一个 URL 列表，要下载 100 个网页的内容。用刚学的 requests，最直接的做法：

```
import time

import requests

urls = ["https://example.com"] * 100 # 简化为相同 URL，但仍是 100 次请求
start = time.time()

for url in urls:
    try:
        r = requests.get(url, timeout=10)

        print(r.status_code)

    except:
        pass

print(f"耗时: {time.time() - start:.2f} 秒")
```

在我的网络上大约 15 秒左右。每次请求的大部分时间都在等待网络数据——发出请求后，CPU 几乎闲置，傻等着服务器回复。这就是 I/O 密集型任务的特点。

如果能让多个请求同时发出，CPU 在等待 1 号请求时可以去处理 2 号请求的返回，效率大幅提升。这就是并发。

Python 并发有三种主流方案：

多线程（Threading）：适合 I/O 密集型，但受限于 GIL（全局解释器锁）

多进程（Multiprocessing）：绕过 GIL，适合 CPU 密集型，开销大

异步 IO（asyncio）：单线程内协程切换，最适合高并发网络请求

我们逐一了解。

13.4 多线程：让多个任务“看起来同时”

线程的基本概念

线程是操作系统能调度的最小执行单元。一个进程可以包含多个线程，共享内存空间。当程序有多个线程时，操作系统会快速在它们之间切换，让它们“看起来”在同时运行。

Python 的 `threading` 模块提供了多线程支持：

```
import threading
import time

def worker(name, delay):
    for i in range(3):
        time.sleep(delay)
        print(f'{name}: 第{i+1}次干活")

# 创建线程
t1 = threading.Thread(target=worker, args=("线程 1", 0.5))
t2 = threading.Thread(target=worker, args=("线程 2", 0.3))

t1.start()
t2.start()

# 等待线程结束
t1.join()
t2.join()

print("全部完成")
```

`start()`启动线程，`join()`等待线程结束。`worker` 函数在两个线程里同时运行，输出会交错出现。

用多线程加速网络请求

```
from concurrent.futures import ThreadPoolExecutor
import requests

urls = ["https://example.com"] * 100

def fetch(url):
    try:
        r = requests.get(url, timeout=10)
        return r.status_code
    except:
        return None

start = time.time()

with ThreadPoolExecutor(max_workers=20) as executor:    # 20 个线程
```

```
results = list(executor.map(fetch, urls))

print(f"耗时: {time.time() - start:.2f} 秒")

print(f"成功: {results.count(200)}")
```

耗时可以降到 2~3 秒。`ThreadPoolExecutor` 自动管理线程池，我们只需提交任务。`max\_workers` 控制同时运行的线程数，对于 I/O 密集型，设置为 20~50 通常效果不错。

新手坑位：多线程容易出资源竞争。

如果多个线程同时修改一个共享变量，结果可能会乱。`threading.Lock` 可以保护临界区，但在我们这类“每个任务独立”的场景下通常不需要。初学阶段，如果多线程任务之间有共享可变状态，优先考虑用队列或避免共享。

GIL: Python 多线程的阿喀琉斯之踵

Python (CPython, 官方实现) 有一个全局解释器锁 (GIL)。它保证同一时刻只有一个线程执行 Python 字节码。也就是说，多线程在 Python 里并不能真正利用多核 CPU 并行计算。对于纯 Python 计算的加速，多线程是无效的。

但对于 I/O 密集型任务（网络、文件读写、数据库），GIL 的影响不大——因为线程在等待 I/O 时会释放 GIL，让其他线程跑。

那如果我要做大量数学计算，真正利用多核怎么办？用多进程。

13.5 多进程：绕过 GIL，真正并行

进程拥有独立的内存空间。多进程可以同时运行在多个 CPU 核心上，不受 GIL 的束缚。但进程间通信比线程间慢，启动一个进程的开销也比线程大。

multiprocessing 模块用法和 threading 类似：

```
from multiprocessing import Process

import os

def worker(name):

    print(f"{name} 运行在进程 {os.getpid()}")

p1 = Process(target=worker, args=("进程 1",))
p2 = Process(target=worker, args=("进程 2",))

p1.start()

p2.start()

p1.join()

p2.join()
```

也有 Pool（池）来方便管理：

```
from multiprocessing import Pool

import time
```

```
def heavy_computation(x):
    return x ** x

with Pool(processes=4) as pool:
    results = pool.map(heavy_computation, range(1000))
```

对于 CPU 密集型任务（科学计算、图像处理），多进程是加速的首选。但从网络请求角度来说，多进程开销太大，不如多线程和异步。

13.6 异步 IO (asyncio): 单线程的高并发王者

多线程虽然有效，但切换线程有系统开销，而且多线程代码容易写出竞态条件。Python 从 3.5 开始引入 `async/await` 语法，提供了一种更优雅的并发模型——协程。

核心思想：一个单线程里运行多个协程，当某个协程需要等待（比如网络请求），它主动交还控制权，让其他协程继续跑。这完全在 Python 内部实现，没有系统调度的开销。非常适合高并发 I/O（网络服务、爬虫）。

async / await 基本语法

`async def` 定义一个协程函数。调用它不执行，而是返回一个协程对象。

`await` 暂停当前协程，等待一个可等待对象（比如网络请求的 `Future`）。

协程必须由事件循环调度运行。

```
import asyncio

async def say_hello(name, delay):
    await asyncio.sleep(delay)  # 模拟 I/O 等待，注意不是 time.sleep
    print(f"你好, {name}!")

async def main():
    # 同时启动多个协程
    await asyncio.gather(
        say_hello("张三", 1),
        say_hello("李四", 0.5),
        say_hello("王五", 0.3),
    )

asyncio.run(main())  # 启动事件循环
```

输出顺序：王五、李四、张三（等待最短的最先完成）。三个协程并发运行。

新手坑位：协程里不能用 `time.sleep()`，它会阻塞整个线程。

必须用 `await asyncio.sleep()` 才能非阻塞等待。

异步网络请求：aiohttp

`requests` 是同步库，不能在协程里用。我们需要异步的 HTTP 库，最流行的是 `aiohttp`。安装：

```
python -m pip install aiohttp
```

用异步并发下载 100 个 URL：

```
import asyncio
import aiohttp
import time

async def fetch(session, url):
    try:
        async with session.get(url, timeout=10) as response:
            return response.status
    except:
        return None

async def main():
    urls = ["https://example.com"] * 100
    async with aiohttp.ClientSession() as session:
        tasks = [fetch(session, url) for url in urls]
        results = await asyncio.gather(*tasks)
        print(f"成功: {results.count(200)}")

start = time.time()
asyncio.run(main())
print(f"耗时: {time.time() - start:.2f} 秒")
```

在我的网络中，耗时 0.8 秒左右，比多线程的 2~3 秒更快，而且代码没有锁或竞态条件的困扰。

13.7 实战：一个异步网络爬虫

我们来写一个实用的异步爬虫：从 `quotes.toscrape.com`（一个专门用于爬虫练习的网站）抓取名言数据，存入 CSV。

该网站有多页，每页有若干条名言（quote）、作者、标签。

```
import asyncio
import aiohttp
import pandas as pd

from bs4 import BeautifulSoup # 需要 pip install beautifulsoup4

BASE_URL = "http://quotes.toscrape.com"
```

```

async def fetch(session, url):
    async with session.get(url) as response:
        return await response.text()

async def parse_page(session, page_num):
    """抓取并解析一页，返回该页的所有名言列表"""
    url = f"{BASE_URL}/page/{page_num}/"
    html = await fetch(session, url)
    soup = BeautifulSoup(html, "html.parser")

    quotes = []
    for quote_div in soup.find_all("div", class_="quote"):
        text = quote_div.find("span", class_="text").text
        author = quote_div.find("small", class_="author").text
        tags = [tag.text for tag in quote_div.find_all("a", class_="tag")]
        quotes.append({"text": text, "author": author, "tags": ", ".join(tags)})
    print(f"第 {page_num} 页抓取完毕, {len(quotes)} 条名言")
    return quotes

async def main():
    async with aiohttp.ClientSession() as session:
        # 先抓第 1 页，获取总页数
        html = await fetch(session, BASE_URL)
        soup = BeautifulSoup(html, "html.parser")
        # 假设最多 10 页，实际可从分页标签提取
        total_pages = 10
        # 并发抓取所有页
        tasks = [parse_page(session, i) for i in range(1, total_pages + 1)]
        pages_quotes = await asyncio.gather(*tasks)
        # 合并所有页的数据
        all_quotes = []
        for page_quotes in pages_quotes:

```

```

    all_quotes.extend(page_quotes)

# 用 Pandas 保存

df = pd.DataFrame(all_quotes)

df.to_csv("quotes.csv", index=False, encoding="utf-8-sig")

print(f"共抓取 {len(all_quotes)} 条名言，已保存到 quotes.csv")

asyncio.run(main())

```

这个爬虫完全异步，10 页几乎同时抓取，耗时极短。同时，我们用了 BeautifulSoup 库来解析 HTML（`pip install beautifulsoup4`），它让从网页中提取数据变得像查字典一样简单。

新手坑位：爬虫要礼貌。

并发请求对目标网站造成的压力是同步的数十倍。真实环境中要控制并发数（使用 `asyncio.Semaphore` 限制同时活跃的协程数）、加请求间隔、遵守 `robots.txt`、最好使用官方 API。我们这里是爬练习网站，没关系。

13.8 总结与下一步

这一章，你的程序从单机跨越到了互联网，并学会了同时做很多事：

requests 深度使用：timeout、异常处理、流式下载。

多线程：适合 I/O 密集，受 GIL 局限，用 `ThreadPoolExecutor` 很简单。

多进程：绕过 GIL，适合 CPU 密集，用 `ProcessPoolExecutor`。

异步 IO：asyncio + aiohttp，单线程高并发 I/O，速度和优雅兼得。

实战爬虫：异步抓取 + HTML 解析 + 数据存储。

现在，你的 Python 技能树已经非常繁茂。但还有一个关键能力没涉及——把数据长久保存起来，做结构化查询。下一章，我们将学习数据库与存储：SQLite、SQLAlchemy、以及何时用 Redis 做缓存。届时，你的爬虫可以把数据存入真正的数据库，而不是 CSV。

练习

1. 用 requests 向 <https://httpbin.org/delay/2> 发 GET 请求（这个接口会延迟 2 秒返回），加上 timeout 和异常处理。

2. 将第 1 题改为并发发出 5 个请求，分别用 `ThreadPoolExecutor` 和 `asyncio + aiohttp` 实现，比较耗时。

3. 写一个异步爬虫，抓取 <https://api.github.com/search/repositories?q=python> 返回的仓库前十名，打印项目名称和星数。（提示：`json()` 返回字典）

4. 解释：什么是 GIL？它主要影响哪种类型的任务？

5. 用 `asyncio.Semaphore` 修改本章爬虫，限制同时最多 3 个并发请求。（提示：async with sem）

练习参考答案

练习 1


```

import requests

try:

    r = requests.get("https://httpbin.org/delay/2", timeout=5)

    r.raise_for_status()

    print(r.json())

except requests.exceptions.Timeout:

    print("请求超时")

except requests.exceptions.RequestException as e:

    print(f"错误: {e}")

```

练习 2

使用多线程:

```

from concurrent.futures import ThreadPoolExecutor

import time, requests

url = "https://httpbin.org/delay/2"

def fetch(url):

    r = requests.get(url, timeout=10)

    return r.status_code

start = time.time()

with ThreadPoolExecutor(max_workers=5) as exe:

    list(exe.map(fetch, [url]*5))

print(f"多线程耗时: {time.time()-start:.2f} 秒") # 约 2 秒

```

使用异步:

```

import asyncio, aiohttp, time

async def fetch(session, url):

    async with session.get(url) as resp:

        return resp.status

async def main():

    url = "https://httpbin.org/delay/2"

    async with aiohttp.ClientSession() as session:

        tasks = [fetch(session, url) for _ in range(5)]

        await asyncio.gather(*tasks)

```

```
start = time.time()

asyncio.run(main())

print(f"异步耗时: {time.time()-start:.2f} 秒") # 也是约 2 秒
```

同步顺序执行需 10 秒，并发只需 2 秒。

练习 3

```
import aiohttp, asyncio

async def main():

    url = "https://api.github.com/search/repositories?q=python"

    async with aiohttp.ClientSession() as session:

        async with session.get(url) as resp:

            data = await resp.json()

            for repo in data["items"][:10]:

                print(f"{repo['name']}: {repo['stargazers_count']} 星")

asyncio.run(main())
```

练习 4

GIL（全局解释器锁）是 CPython 解释器的设计取舍，它保证同一时刻只有一个线程执行 Python 字节码。主要影响 CPU 密集型任务的多线程加速（多线程无法利用多核），对 I/O 密集型任务影响较小（等待 I/O 时释放 GIL）。可用多进程或 C 扩展绕过。

练习 5

```
async def parse_page_with_limit(session, page_num, sem):

    async with sem: # 限制并发数

        return await parse_page(session, page_num)

# 在 main() 中:

sem = asyncio.Semaphore(3)

tasks = [parse_page_with_limit(session, i, sem) for i in range(1, 11)]
```

第 14 章 数据库与存储：把数据留住

你的爬虫抓下了几万条数据，存在 CSV 里。一开始还好，但当你需要查“作者是鲁迅且标签包含‘爱情’的名言”时，你打开 CSV 用 Pandas 过滤——似乎还行。当数据变成几百万行，查询条件变复杂，还要同时被多个程序访问时，CSV 就会崩溃。

数据库就是专门解决“长久保存、高效查询、并发访问”问题的系统。这一章，我们从零

学习数据库思维，然后实操 Python 操作两种主流数据库：关系型数据库 SQLite 和非关系型数据库 Redis。最后我们用 SQLAlchemy——一个能把数据库表当成 Python 类来用的神器，提升开发效率。

学完这一章，你的数据就有了真正的“家”。

14.1 为什么需要数据库？

文件存储（CSV、JSON）有几个致命弱点：

查询效率：要找一条特定记录，必须遍历整个文件。

并发写入：两个程序同时写一个文件，数据可能错乱。

数据一致性：如果程序崩溃，文件可能只写了一半，数据损坏。

结构化关系：学生和课程（多对多关系）在 CSV 里很难表达。

数据库系统（DBMS）把这些问题全包揽了：它用高效的索引加速查询，用事务确保数据完整性，用锁机制协调并发。你只需要用结构化语言告诉它“做什么”，不用管“怎么实现”。

14.2 SQLite：自给自足的轻量数据库

SQLite 是一个嵌入式的数据库——它不需要安装服务器，整个数据库就是一个 .db 文件。Python 标准库自带 sqlite3 模块，开箱即用。适合个人项目、移动应用、桌面软件和教学。

连接与创建表

```
import sqlite3

# 连接数据库（如果文件不存在会自动创建）

conn = sqlite3.connect("school.db")

cursor = conn.cursor()

# 创建一张学生表

cursor.execute("""

    CREATE TABLE IF NOT EXISTS students (

        id INTEGER PRIMARY KEY AUTOINCREMENT,

        name TEXT NOT NULL,

        age INTEGER,

        score REAL

    )

""")

conn.commit()
```

cursor 是游标，执行 SQL 语句并获取结果。

SQL 语句不区分大小写，但惯例关键字大写，表名列名小写。

IF NOT EXISTS 防止重复创建表时报错。

PRIMARY KEY 是主键，唯一标识一行，通常自增。

NOT NULL 约束该列不能为空。

增删改查（CRUD）

插入数据：

```
# 单条插入

cursor.execute("INSERT INTO students (name, age, score) VALUES (?, ?, ?)", ("张三", 18, 88))

# 批量插入

students = [("李四", 19, 92), ("王五", 18, 67), ("赵六", 20, 74)]

cursor.executemany("INSERT INTO students (name, age, score) VALUES (?, ?, ?)", students)

conn.commit()
```

新手坑位：绝对不要用字符串拼接构造 SQL。

比如 `f"INSERT INTO ... VALUES ('{name}', {age})"` 会引发 SQL 注入漏洞——当 `name` 里包含恶意 SQL 语句时，整个数据库可能被清空。务必使用参数化占位符？。

查询数据：

```
# 查询所有

cursor.execute("SELECT * FROM students")

rows = cursor.fetchall()

for row in rows:

    print(row)    # (1, '张三', 18, 88.0)

# 条件查询

cursor.execute("SELECT name, score FROM students WHERE score >= ?", (80,))

print(cursor.fetchall())

# 排序与限制

cursor.execute("SELECT * FROM students ORDER BY score DESC LIMIT 3")
```

`fetchall()` 返回所有结果行的列表。

`fetchone()` 返回第一行。

结果以元组形式返回，顺序与建表时的列顺序一致。

更新与删除：

```
cursor.execute("UPDATE students SET score = ? WHERE name = ?", (95, "张三"))
cursor.execute("DELETE FROM students WHERE id = ?", (5,))
conn.commit()
```

使用 with 语句自动提交/回滚

可以用 conn 作为上下文管理器，代码块结束时自动提交，如果发生异常则回滚：

```
with sqlite3.connect("school.db") as conn:
    conn.execute("INSERT INTO students (name, age, score) VALUES (?, ?, ?)", ("
钱七", 17, 81))
# 此处已自动提交
```

简单但实用：写一个学生查询小工具

```
def query_by_name(db_path, name):
    with sqlite3.connect(db_path) as conn:
        cursor = conn.execute(
            "SELECT name, age, score FROM students WHERE name LIKE ?",
            (f"%{name}%",)
        )
        return cursor.fetchall()
print(query_by_name("school.db", "张"))
```

LIKE 和%实现模糊匹配。

14.3 SQLAlchemy：用 Python 类代替 SQL 字符串

手写 SQL 虽然直接，但代码中夹杂大量字符串，难维护，字段名写错也只会运行时才知道。对象关系映射（ORM）让我们用 Python 类来表达数据库表，用方法调用来替换 SQL。SQLAlchemy 是 Python 里最强大的 ORM 库。

安装

```
python -m pip install sqlalchemy
```

定义模型与创建表

```
from sqlalchemy import create_engine, Column, Integer, String, Float
from sqlalchemy.orm import declarative_base, sessionmaker
# 数据库连接
engine = create_engine("sqlite:///school.db", echo=False) # echo=True 可看生成
的 SQL
Base = declarative_base()
```

```

class Student(Base):
    __tablename__ = "students"

    id = Column(Integer, primary_key=True, autoincrement=True)

    name = Column(String, nullable=False)

    age = Column(Integer)

    score = Column(Float)

    def __repr__(self):
        return f"<Student(id={self.id}, name='{self.name}', score={self.score})>"

# 创建表（如果已存在且结构一致，不会出错）
Base.metadata.create_all(engine)

```

一张表就是一个类，列就是类的属性。代码即文档，IDE 能有智能提示。

使用 Session 进行 CRUD

SQLAlchemy 通过 session 管理所有数据库操作：

```

Session = sessionmaker(bind=engine)
session = Session()

```

增加：

```

s1 = Student(name="张三", age=18, score=88)
s2 = Student(name="李四", age=19, score=92)
session.add(s1)
session.add_all([s2, Student(name="王五", age=18, score=67)])
session.commit()

```

直接操作 Python 对象，会话追踪改动，commit()时一次性写入。

查询：

```

# 查全部
students = session.query(Student).all()

# 条件过滤
high_scores = session.query(Student).filter(Student.score >= 80).all()

# 获取第一个
zhang = session.query(Student).filter(Student.name == "张三").first()
print(zhang)

```

排序和分页

```
top3 = session.query(Student).order_by(Student.score.desc()).limit(3).all()
```

查询语法是链式调用，非常直观。

更新：

```
zhang = session.query(Student).filter(Student.name == "张三").first()
```

```
zhang.score = 95
```

```
session.commit() # 直接修改属性，提交后生效
```

删除：

```
wang = session.query(Student).filter(Student.name == "王五").first()
```

```
session.delete(wang)
```

```
session.commit()
```

关系：学生、课程与成绩（多对多）

关系型数据库的精华是关系。假设每个学生可以选多门课程，每门课有多个学生，成绩存在关联表里。SQLAlchemy 能轻松建模：

```
from sqlalchemy import Table, ForeignKey
```

```
from sqlalchemy.orm import relationship
```

中间表（多对多）

```
enrollments = Table(
```

```
    "enrollments",
```

```
    Base.metadata,
```

```
    Column("student_id", ForeignKey("students.id"), primary_key=True),
```

```
    Column("course_id", ForeignKey("courses.id"), primary_key=True),
```

```
    Column("grade", Float),
```

```
)
```

```
class Course(Base):
```

```
    __tablename__ = "courses"
```

```
    id = Column(Integer, primary_key=True)
```

```
    name = Column(String, nullable=False)
```

```
    students = relationship("Student", secondary=enrollments,
back_populates="courses")
```

```
class Student(Base):
```

```
    __tablename__ = "students"
```

```

    id = Column(Integer, primary_key=True, autoincrement=True)

    name = Column(String, nullable=False)

    age = Column(Integer)

    courses = relationship("Course", secondary=enrollments,
back_populates="students")

Base.metadata.create_all(engine)

现在你可以直接通过 Python 对象关联:

s1 = Student(name="张三")

c1 = Course(name="Python 编程")

s1.courses.append(c1) # 选课

session.add_all([s1, c1])

session.commit()

# 查询张三的所有课程

for course in s1.courses:

    print(course.name)

```

不需要写一条 JOIN 语句, SQLAlchemy 帮你完成。这就是 ORM 的生产力。

新手坑位: 查询后别忘记关闭 session 或使用 with。

一般用 with sessionmaker(...).as_session() 自动关闭。长时间不关会占用连接。

14.4 Redis: 超快的缓存和临时数据

有时候你不需要数据永久存储, 但要极快读写、支持过期、排行榜、消息队列等功能。关系型数据库能做到但速度瓶颈明显。Redis 是一种内存数据库, 数据存在内存中, 读写速度微秒级。

Python 用 redis-py 操作 Redis:

```
python -m pip install redis
```

需要先在本地装 Redis 服务, 对于新手建议安装社区维护版:

1. 下载与解压: 从 GitHub 的 tporadowski/redis 项目页面下载最新 ZIP 包并解压, 建议将解压后的 redis-server.exe 等文件放在 C:\redis 目录下。

2. 临时启动与测试:

直接双击 redis-server.exe, 看到 * Ready to accept connections 的提示后, 表明启动成功。

保持此窗口, 双击 redis-cli.exe 打开客户端, 输入命令 ping, 如果返回 PONG, 说明客户端已成功连接服务端, 可以开始使用了。

常用服务管理命令:

启动服务：net start redis。

停止服务：net stop redis。

卸载服务：redis-server --service-uninstall。

也可以通过 redis-server --service-start 和 redis-server --service-stop 来管理。

基础键值操作

```
import redis

r = redis.Redis(host="localhost", port=6379, decode_responses=True)

r.set("username", "张三")

print(r.get("username")) # 张三

r.set("temp", "60 秒后消失", ex=60) # 设置过期时间
```

decode_responses=True 避免返回值全是 bytes。

哈希、列表、集合、有序集合

Redis 不只是字符串键值，它提供了丰富的数据结构：

```
# 哈希

r.hset("user:1", mapping={"name": "张三", "age": 18})

print(r.hgetall("user:1"))

# 列表（做消息队列）

r.lpush("queue", "task1", "task2")

print(r.rpop("queue")) # task1

# 集合（标签）

r.sadd("tags:article1", "Python", "数据库")

print(r.smembers("tags:article1"))

# 有序集合（排行榜）

r.zadd("scores", {"张三": 88, "李四": 92})

print(r.zrevrange("scores", 0, -1, withscores=True))
```

Redis 特别适合：

缓存：存储 API 调用结果，设置几分钟过期，减少后端压力。

会话管理：Web 应用的登录状态。

计数器：点赞数、阅读量。

消息队列：用列表实现生产-消费模型。

14.5 实战：把爬虫数据存入数据库并查询

我们在第 13 章写了一个爬虫，抓了名言数据存为 CSV。现在改成存入 SQLite 数据库，然后用 SQLAlchemy 做一个查询服务。

1. 定义模型

```
from sqlalchemy import create_engine, Column, Integer, String, Text
from sqlalchemy.orm import declarative_base, sessionmaker

engine = create_engine("sqlite:///quotes.db")

Base = declarative_base()

class Quote(Base):
    __tablename__ = "quotes"

    id = Column(Integer, primary_key=True)

    text = Column(Text)

    author = Column(String(100))

    tags = Column(String(200)) # 逗号分隔的标签

Base.metadata.create_all(engine)

Session = sessionmaker(bind=engine)
```

2. 爬虫存入数据库

只需在爬虫解析函数中，把每条名言用 ORM 对象存入：

```
# 在 parse_page 协程里，拿到 quotes 列表后

def save_to_db(quotes):

    with Session() as session:

        for q in quotes:

            session.add(Quote(**q))

        session.commit()
```

3. 查询服务

```
def query_quotes(author=None, tag=None, limit=10):

    with Session() as session:

        q = session.query(Quote)

        if author:

            q = q.filter(Quote.author.like(f"%{author}%"))

        if tag:

            q = q.filter(Quote.tags.contains(tag))
```

```
return q.limit(limit).all()
```

轻松实现多维组合查询。SQLAlchemy 的 filter 可链式叠加，这比拼字符串 SQL 强太多。

14.6 总结与下一步

这一章，你学会了给数据一个结构化的家：

SQLite：零安装的内嵌数据库，用标准库 sqlite3 执行 SQL。

SQLAlchemy：将数据库表映射为 Python 类，用面向对象的方式操作数据库，支持复杂关系。

Redis：内存数据库，极速读写，擅长缓存、计数、队列。

现在是时候让你的程序在互联网上真正“活”起来——下一章，我们进入 Web 应用与 API 设计。你将学会用 FastAPI 快速搭建 Web 服务，把数据库里查到的数据通过 HTTP API 暴露给前端或任何客户端调用。你的爬虫和分析成果，将变成能对外服务的产品。

练习

1. 使用 sqlite3 创建一个图书表 books (id, title, author, year)，插入 3 条记录，查询并打印所有记录。

2. 将第 1 题改用 SQLAlchemy 实现（定义模型、创建表、插入、查询）。

3. 实现一个函数 update_score(db_path, name, new_score)，用 sqlite3 更新指定学生的分数。

4. 假设有一万个键值对要缓存，每个缓存 2 分钟后自动过期，用 Redis 的 mset 与 expire 吗？请说明如何以合理方式高效设置过期，写出伪代码。

5. 用 SQLAlchemy 的 relationship 定义“作者-书籍”一对多关系（Author 有多个 Book），并写代码创建一个作者并添加两本书。

练习参考答案

练习 1

```
import sqlite3

with sqlite3.connect("library.db") as conn:

    conn.execute("CREATE TABLE IF NOT EXISTS books (id INTEGER PRIMARY KEY AUTOINCREMENT, title TEXT, author TEXT, year INTEGER)")

    conn.execute("INSERT INTO books (title, author, year) VALUES (?, ?, ?)", ("三体", "刘慈欣", 2008))

    conn.execute("INSERT INTO books (title, author, year) VALUES (?, ?, ?)", ("活着", "余华", 1993))

    conn.execute("INSERT INTO books (title, author, year) VALUES (?, ?, ?)", ("百年孤独", "马尔克斯", 1967))

    rows = conn.execute("SELECT * FROM books").fetchall()

    for r in rows:
```

```
print(r)
```

练习 2

```
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import declarative_base, sessionmaker

engine = create_engine("sqlite:///library.db")

Base = declarative_base()

class Book(Base):
    __tablename__ = "books"

    id = Column(Integer, primary_key=True)

    title = Column(String)

    author = Column(String)

    year = Column(Integer)

Base.metadata.create_all(engine)

Session = sessionmaker(bind=engine)

with Session() as session:

    session.add_all([

        Book(title="三体", author="刘慈欣", year=2008),

        Book(title="活着", author="余华", year=1993),

        Book(title="百年孤独", author="马尔克斯", year=1967),

    ])

    session.commit()

    for book in session.query(Book).all():

        print(book.title, book.author)
```

练习 3

```
def update_score(db_path, name, new_score):

    with sqlite3.connect(db_path) as conn:

        conn.execute("UPDATE students SET score = ? WHERE name = ?", (new_score,
name))

        if conn.total_changes == 0:

            print("未找到该学生")

        else:

            print("更新成功")
```

练习 4

Redis 的 SET 命令支持直接带过期参数 EX, 但 MSET 不支持批量设过期。高效做法: 用 SET key value EX 120 逐个执行, 或使用 pipeline 减少网络往返:

```
import redis

r = redis.Redis()

pipe = r.pipeline()

for i in range(10000):

    pipe.set(f"cache:{i}", f"value{i}", ex=120)

pipe.execute()
```

pipeline 会将多条命令打包一次发送, 避免一万次网络 RTT。

练习 5

```
from sqlalchemy import ForeignKey
from sqlalchemy.orm import relationship

class Author(Base):

    __tablename__ = "authors"

    id = Column(Integer, primary_key=True)

    name = Column(String, nullable=False)

    books = relationship("Book", back_populates="author")

class Book(Base):

    __tablename__ = "books"

    id = Column(Integer, primary_key=True)

    title = Column(String)

    author_id = Column(Integer, ForeignKey("authors.id"))

    author = relationship("Author", back_populates="books")

Base.metadata.create_all(engine)

Session = sessionmaker(bind=engine)

with Session() as session:

    author = Author(name="刘慈欣")

    author.books = [Book(title="三体"), Book(title="流浪地球")]

    session.add(author)

session.commit()

print([b.title for b in author.books])
```

第 15 章 测试、调试与代码质量：写给人看的代码

到这里，你已经能写出功能完整的程序。但程序越写越大，一个残酷的真相会慢慢浮现：写代码的时间只占 20%，剩下的 80% 都在调试、改 bug、读代码。

代码是写给人看的，顺便能在机器上运行。一个程序的生命周期里，你会反复读它、改它。如果代码乱成一团，每次改动都是折磨；如果连你自己都看不懂自己三个月前的逻辑，这个程序其实已经死了。

这一章，我们暂时不学新功能，而是学如何写出让人放心的代码——调试错误的根源、用测试保护已有功能、用类型注解消灭低级错误、用风格规范让代码整洁如新。

15.1 调试的艺术：问题出在哪？

程序报错是家常便饭。SyntaxError 还好，Python 直接告诉你哪行写错了。最头疼的是逻辑错误——程序能跑，但结果不对，没有任何报错。这种时候，你需要调试。

15.1.1 最朴素也最有效：print 大法

最简单粗暴的调试手段是加 print()，在关键位置打印变量值：

```
def calculate_bmi(weight, height):  
  
    bmi = weight / (height ** 2)  
  
    print(f"DEBUG: weight={weight}, height={height}, bmi={bmi}") # 调试  
  
    if bmi < 18.5:
```

```

        return "过轻"

    elif bmi < 24:

        return "正常"

    elif bmi < 28:

        return "超重"

    else:

        return "肥胖"

result = calculate_bmi(70, 1.75)

print(result)

```

优点：零学习成本，立刻能用。缺点：看完了要删，删漏了很尴尬，而且对于复杂程序，`print` 可能淹没在海量输出里。

不过，不要轻视 `print`。很多资深工程师在排查紧急线上问题时首选依然是 `print` 或日志，因为它不侵入运行态。只是日常开发中，我们有更强大的工具。

15.1.2 更专业的断点调试：pdb

Python 内置了一个交互式调试器 `pdb`。它能在你指定的行暂停程序，让你像时间被冻结一样，检查当前所有变量的值。

在你的代码里插入一行 `breakpoint()` (Python 3.7+)，当程序执行到这一行时，会进入 `pdb` 交互模式：

```

def divide(a, b):

    breakpoint()    # 程序在这里暂停

    return a / b

result = divide(10, 0)

```

运行这个脚本，终端变成交互式调试器：

```

> /path/to/script.py(4)divide()
-> return a / b

(Pdb) print(a)           # 查看变量
10

(Pdb) print(b)
0

(Pdb) step                # 单步执行（下一步）
ZeroDivisionError: division by zero

```

常用 `pdb` 命令：

命令	简写	作用
list	l	显示当前位置的源代码
<code>print(变量) p</code> 变量 打印变量值		
next	n	执行下一行（不进入函数内部）
step	s	单步执行（进入函数内部）
continue	c	继续运行直到下一个断点
quit	q	退出调试

新手坑位：breakpoint()留到生产代码里会让程序卡住。

调试完记得删除所有 breakpoint()。好在 Python 3.7+ 可以通过设置环境变量 PYTHONBREAKPOINT=0 全局禁用所有 breakpoint，上线前加一层保障。

15.1.3 IDE 断点调试：最舒适的方案

如果你用 VSCode 或 PyCharm，其实不需要手动加 breakpoint()。在编辑器中，在行号左边点一下就会出现一个红点（断点），然后以调试模式运行脚本（VSCode 点击左上角“运行和调试”），程序会在红点处暂停。右侧面板会显示当前所有变量的值，你可以逐步执行、查看调用栈。

这是最直观的调试方式，强烈建议你学会。VSCode 官网有五分钟快速入门视频，这里不截图，但你要知道这比 print 高效十倍。

15.2 测试：给程序上保险

改了代码，怎么知道以前正常的功能没被破坏？运行一次所有功能？手动点一遍各个界面？这种人工回归测试既慢又漏，而且人会疏忽。

自动化测试就是给程序写“测试用例”，每次改动后跑一遍，机器告诉你哪些通过、哪些失败。Python 有强大的测试框架 pytest。

安装：

```
python -m pip install pytest
```

15.2.1 写第一个测试

假设你有一个 calculator.py 模块：

```
# calculator.py
def add(a, b):
    return a + b
def divide(a, b):
```



```
if b == 0:

    raise ValueError("除数不能为零")

return a / b
```

测试文件命名为 test_calculator.py (pytest 默认找以 test_开头的文件):

```
# test_calculator.py

import pytest

from calculator import add, divide

def test_add():

    assert add(2, 3) == 5

    assert add(-1, 1) == 0

    assert add(0, 0) == 0

def test_divide_basic():

    assert divide(10, 2) == 5.0

    assert divide(7, 2) == 3.5

def test_divide_by_zero_raises():

    with pytest.raises(ValueError, match="除数不能为零"):

        divide(10, 0)
```

然后在终端运行:

```
pytest
```

输出会告诉你几个测试通过、几个失败。如果有失败的,会精确显示哪个断言失败了,预期值是什么,实际值是什么。

assert 关键字在测试里就是“断言”——如果后面的表达式为 False,测试失败。pytest.raises 用来断言某段代码会抛出指定异常。

15.2.2 测试驱动开发 (TDD) 的精神

测试不是写完后补的,而应该是先写测试,再写功能。流程:

- 1.写一个测试,描述你期待的功能(此时测试肯定失败,因为功能还没写)
- 2.写最少的代码让这个测试通过
- 3.重构代码,让结构更好(测试会保护你不改坏)
- 4.重复

这就是“红灯 → 绿灯 → 重构”循环。初学不用完全遵守,但要知道这种思维:把“功能能用了”变成“功能有证明文件了”。

15.3 代码风格：统一是最大的可读性

你有过看一本翻译极烂的外国小说吗？每段话都能懂，但读几页就头疼。代码也是一样：混乱的命名、不一致的空格、缺失的注释，都会让阅读变成折磨。

Python 社区有统一的风格指南 PEP 8。好消息是你不用一条条背——有自动化工具帮你检查甚至自动修复。

15.3.1 核心原则速览

缩进：4 个空格，不用 Tab。

每行最长 79 字符（文档 72 字符）。现在宽屏时代很多人放宽到 120，但团队内要统一。

函数和类之间空两行，方法之间空一行。

命名：

普通变量和函数：snake_case（全小写，下划线分隔）

类名：CapWords（大驼峰）

常量：ALL_CAPS

比较：和 None 比较用 is 和 is not，不用 ==。

布尔值：不要用 if x == True，直接写 if x。

导入：每个导入单独一行，按标准库→第三方库→本地模块的顺序分组。

15.3.2 用工具自动检查和修复

flake8 检查风格违规：

```
python -m pip install flake8
flake8 your_file.py    # 列出所有违规行
```

black 自动格式化代码成标准风格：

```
python -m pip install black
black your_file.py    # 直接改写文件，让它符合规范
```

black 的哲学是“风格就是统一”，它的部分决定你可能不完全认同（比如它偏爱双引号），但不需要争论——直接用它，全社区都用它，争议就消失了。

用 VSCode 的话，可以在设置里配置保存时自动运行 black，从此你的代码永远干净。

15.4 类型注解：给函数贴上“说明书”

Python 是动态类型语言，变量可以指向任何类型的对象。灵活是灵活，但代码量大了以后，你经常搞不清一个函数接收什么类型、返回什么类型。调用者可能传入错误类型，在运行时才报错。

从 Python 3.5 开始，你可以给函数加上类型注解——不强制类型，但作为“文档”和静态检查工具的依据。

15.4.1 基本注解语法

```
def greet(name: str) -> str:
    return f"你好, {name}!"

def calculate_bmi(weight: float, height: float) -> float:
    return weight / (height ** 2)

number: int = 42  # 变量也可以注解

names: list[str] = ["张三", "李四"]  # Python 3.9+
```

函数参数后面加: 类型, 返回类型加-> 类型。这些注解不影响运行时, Python 解释器完全忽略它们。

15.4.2 联合类型和 Optional

有时一个参数可以有多种类型:

```
from typing import Union, Optional

def process(value: Union[int, str]) -> str:
    return str(value)

# Optional[X] 等价于 Union[X, None]

def get_username(user_id: int) -> Optional[str]:
    if user_id == 1:
        return "admin"

    return None
```

Python 3.10+ 可以用 `int | str` 代替 `Union[int, str]`, 用 `str | None` 代替 `Optional[str]`。本书用 3.10 版本, 所以可以直接用竖线写法。

15.4.3 用 mypy 做静态检查

类型注解的价值在于配合 mypy 做静态类型检查——在运行前就发现类型错误。

```
python -m pip install mypy
```

写一段有类型错误的代码:

```
def add(a: int, b: int) -> int:
    return a + b

result: str = add(3, 5)  # 返回值是 int, 却赋给了 str
```

运行 mypy:

```
mypy script.py
```

它会报告: `error: Incompatible types in assignment (expression has type "int", variable has type`

```
"str")
```

这对于大型项目非常重要——重构后运行 mypy，能帮你找到所有类型不匹配的地方，比运行时翻异常快得多。

原则：类型注解是可选的，不是必须的，但在函数签名、公共 API、大项目中大力推荐。初学时可以不写，但读别人代码要能认出来。

15.5 实战：测试并调试一个小项目

假设我们写了一个“银行账户”模块，支持转账。现在要写测试，并故意埋一个 bug 调试它。

account.py

```
class InsufficientFundsError(Exception):
    pass

class BankAccount:
    def __init__(self, owner: str, balance: float = 0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount: float) -> None:
        if amount <= 0:
            raise ValueError("存款金额必须为正数")
        self.balance += amount

    def withdraw(self, amount: float) -> None:
        if amount <= 0:
            raise ValueError("取款金额必须为正数")
        if amount > self.balance:
            raise InsufficientFundsError(f"余额不足（当前 {self.balance}）")
        self.balance -= amount

    def transfer_to(self, target: "BankAccount", amount: float) -> None:
        self.withdraw(amount)
        target.deposit(amount)
```

test_account.py

```
import pytest

from account import BankAccount, InsufficientFundsError

def test_deposit():
```

```

    acc = BankAccount("张三", 100)

    acc.deposit(50)

    assert acc.balance == 150
def test_deposit_negative_raises():

    acc = BankAccount("张三")

    with pytest.raises(ValueError):

        acc.deposit(-10)
def test_withdraw():

    acc = BankAccount("张三", 100)

    acc.withdraw(40)

    assert acc.balance == 60
def test_withdraw_insufficient_raises():

    acc = BankAccount("张三", 50)

    with pytest.raises(InsufficientFundsError):

        acc.withdraw(100)
def test_transfer():

    alice = BankAccount("张三", 200)

    bob = BankAccount("李四", 100)

    alice.transfer_to(bob, 80)

    assert alice.balance == 120

    assert bob.balance == 180

```

运行 pytest，应该全部通过。

现在，假设我们在 `transfer_to` 里把 `self.withdraw(amount)` 和 `target.deposit(amount)` 调反了，写了 `self.deposit(amount)` 然后 `target.withdraw(amount)`。运行测试会发现 `test_transfer` 失败，且 `alice.balance` 多了 80 而不是少了 80。这时如果你不确定为什么，可以在 `transfer_to` 里临时加 `breakpoint()`，用 `pdb` 单步进去看每一步后余额的变化。

这就是“测试保护你，调试帮你定位”的黄金组合。

15.6 总结与下一步

这一章，你没有学新的编程概念，但学到了程序员赖以生存的元技能：

调试：print 快速检查，`breakpoint()` / IDE 断点深入追踪。

测试：用 `pytest` 写自动化测试，给功能上永久保险。

风格：PEP 8 是标准，`black` 和 `flake8` 帮你自动达标。

类型注解：用类型给函数写“契约”，用 mypy 提前发现错误。

现在，你不仅能写出能用的程序，还能写出可维护、可验证、可协作的程序。

练习

1. 写一个函数 `safe_divide(a, b)`，在 `b == 0` 时返回 `None` 而不是抛异常。用 `pytest` 写三个测试：正常除法、除以 0、负数除法。

2. 在练习 1 的函数中故意引入一个 bug（比如不处理除以 0），然后使用 `breakpoint()` 调试它。

3. 给下面的代码加上类型注解，然后运行 mypy 检查：

```
def multiply(a, b):  
    return a * b  
  
result = multiply(3, "5")  # 应该被检查出来
```

4. 对你的一个已有脚本（比如第 10 章的天气查询工具）运行 `black` 自动格式化，观察前后的格式变化。

5. 运行 `flake8` 检查练习 4 格式化的脚本，看是否还有警告。如果有，手工修复。

练习参考答案

练习 1

`safe_divide.py`

```
def safe_divide(a, b):  
    if b == 0:  
        return None  
    return a / b
```

`test_safe_divide.py`

```
import pytest  
  
from safe_divide import safe_divide  
  
def test_normal_division():  
    assert safe_divide(10, 2) == 5.0  
  
def test_divide_by_zero():  
    assert safe_divide(10, 0) is None  
  
def test_negative_division():  
    assert safe_divide(-10, 2) == -5.0
```

练习 2

例如把 `if b == 0` 错写成 `if b == "0"`，除法会抛异常。在函数开头插入 `breakpoint()`，单步执行观察 `b` 的值和被比较的字符串，会发现类型不匹配。

练习 3

```
def multiply(a: int, b: int) -> int:
    return a * b

result: int = multiply(3, "5") # mypy 会报错
```

练习 4

```
black weather.py

black 会调整换行、引号统一、空格对齐等。
```

练习 5

```
flake8 weather.py
```

常见警告：行太长、模块级导入顺序不对、多余空行、函数间少空行等，按提示逐行修正即可。

第 16 章 架构与设计：从小项目到可维护的系统

你跟着这本书走到了这里。

你能写面向对象的代码，能处理复杂数据。但如果现在交给你一个十万行的项目，让你在上面加一个新功能，你可能会感到无从下手——不是因为语法不熟，而是因为不知道代码是怎么组织起来的。

一个能跑的程序和一个能活十年的系统，中间隔着一道鸿沟，叫软件设计。这一章，我们不学新语法，而是聊聊那些让代码“长寿”的思维方式：设计模式、分层架构、阅读源码的方法，以及从这扇门出去之后，你该如何继续走下去。

16.1 从“能跑”到“好改”：设计原则初探

你前面写的所有程序，功能都对。但如果三个月后需求变了——比如 BMI 计算要换成英制单位，爬虫要换一个网站，Web API 要加用户认证——你改起来痛不痛苦？

痛苦的根源通常是耦合：一个模块的内部细节被另一个模块依赖。改了 A，B 炸了；修了 B，C 又炸了。好的设计，就是让模块之间松耦合，每个模块内部高内聚。

有一组经典的设计原则叫 SOLID，你不用背全，但理解其中两条就够用很久：

单一职责原则：一个类只做一件事。如果你发现一个类里有“计算 BMI”和“发邮件通知用户”两件毫不相干的事，应该拆成两个类。

开闭原则：对扩展开放，对修改关闭。意思是，加新功能时，能不能不修改已有代码，只增加新代码？第 9 章我们把飞行行为抽成组合，就是这个思想的体现——加一种新的飞行方式，只新增一个类，不动 Bird 的代码。

这些是指导方针，不是死规矩。初学时不必强求，但从今天起，每次改代码感到“牵一发动全身”时，问自己：是不是耦合太紧了？

16.2 设计模式在 Python 中的缩影

设计模式是前辈们总结的“常见问题的常见解法”。在 Python 里，很多经典模式因为语言本身的灵活性而变得极其简单，甚至隐于无形。

16.2.1 单例模式：全局只有一个实例

有些对象全程序只需要一个——比如配置管理器、数据库连接池。单例模式保证一个类只能创建一个实例。

Python 实现单例的最简单方式是用模块级变量——模块在 Python 进程中只加载一次，天然单例：

config.py

```
class AppConfig:
    def __init__(self):
        self.debug = False
        self.db_url = "sqlite:///app.db"
config = AppConfig() # 模块加载时创建唯一实例
```

其他文件

```
from config import config
```

不需要复杂的元类和锁。Python 的模块导入机制就是天然的单例工厂。

16.2.2 工厂模式：把“创建什么类”集中管理

当需要根据不同条件创建不同对象时，把创建逻辑抽成一个函数或类，而不是在代码到处 if-else 判断。第 11 章的 `make_multiplier` 就是一个函数工厂。更典型的例子：

```
class Dog:
    def speak(self): return "汪汪"
class Cat:
    def speak(self): return "喵喵"
def animal_factory(animal_type: str):
    if animal_type == "dog":
        return Dog()
    elif animal_type == "cat":
        return Cat()
    else:
```



```
        raise ValueError(f"未知动物类型: {animal_type}")

pet = animal_factory("dog")

print(pet.speak()) # 汪汪
```

以后加新动物，只改工厂函数一处，调用者不变。

16.2.3 观察者模式：状态变了，自动通知所有订阅者

比如一个订单状态从“待支付”变为“已支付”，需要自动发短信、减库存、记日志。观察者模式让状态拥有者把“订阅者”列个清单，状态一变就逐个通知。

Python 用简单的回调函数或列表就能实现：

```
class Order:

    def __init__(self):

        self.status = "待支付"

        self._observers = []

    def attach(self, observer):

        self._observers.append(observer)

    def pay(self):

        self.status = "已支付"

        for observer in self._observers:

            observer(self)

def send_sms(order):

    print(f"[短信] 订单{id(order)}已支付")

def reduce_inventory(order):

    print(f"[库存] 订单{id(order)}库存已扣减")

order = Order()

order.attach(send_sms)

order.attach(reduce_inventory)

order.pay()
```

输出：

```
[短信] 订单 140234... 已支付
[库存] 订单 140234... 库存已扣减
```

没有复杂的 EventListener 类层次，函数就是观察者。

很多设计模式在 Python 里被简化甚至不需要了。但知道它们的名字和意图，能帮你阅读大型框架源码时理解设计思路。

16.3 组织代码：分层结构的魔力

当你写了一个处理数据的小项目，所有逻辑可能都挤在同一个 .py 文件里：读文件、清洗数据、计算统计、打印结果。在三百行的时候还行，三千行的时候就是灾难。

一个能长期维护的项目，通常会把代码按职责分层。最常见的三层是：

用户交互层	← 接收输入，展示结果（命令行参数、print、input）
业务逻辑层	← 核心规则、计算、决策（纯 Python，不依赖文件或数据库）
数据访问层	← 文件读写、数据库操作、缓存

核心规则：每一层只能调用它下面那一层。数据访问层不知道有业务逻辑层，业务逻辑层不知道有用户交互层。

看一个实际例子。假设你有一个“学生成绩管理”的小程序，最初可能是这样写的：

```
# 一团乱麻版本

import csv

def main():

    students = []

    with open("scores.csv", "r") as f:

        reader = csv.DictReader(f)

        for row in reader:

            students.append(row)

    # 计算平均分

    total = 0

    for s in students:

        total += float(s["score"])

    avg = total / len(students)

    # 找出最高分

    max_score = 0

    best_student = ""

    for s in students:

        if float(s["score"]) > max_score:

            max_score = float(s["score"])
```

```

        best_student = s["name"]

    print(f"平均分: {avg:.1f}")

    print(f"最高分: {best_student} ({max_score})")

if __name__ == "__main__":

    main()

```

功能正确,但所有东西搅在一起:怎么读数据、怎么算、怎么展示全写在一块。以后想把 CSV 换成数据库,或者想把结果生成 HTML 表格而不是打印,就得大改代码。

现在我们把它重构为三层结构。

第一步:数据访问层(管数据怎么存取)

`dal.py`

```

import csv

def load_students(filename: str) -> list[dict]:

    """从 CSV 文件读取学生数据, 返回字典列表"""

    students = []

    with open(filename, "r", encoding="utf-8") as f:

        reader = csv.DictReader(f)

        for row in reader:

            row["score"] = float(row["score"]) # 统一转为数字

            students.append(row)

    return students

def save_results(filename: str, results: list[dict]) -> None:

    """将统计结果保存为 CSV 文件"""

    if not results:

        return

    with open(filename, "w", newline="", encoding="utf-8") as f:

        writer = csv.DictWriter(f, fieldnames=results[0].keys())

        writer.writeheader()

        writer.writerows(results)

```

这一层只关心“数据从哪来、到哪去”,不知道数据会被用来算什么。以后换成 SQLite 数据库,只改这两个函数,其他层完全不动。

第二步:业务逻辑层(管核心计算规则)

`service.py`

```

from dal import load_students

def calculate_stats(students: list[dict]) -> dict:
    """计算平均分、最高分、最低分，返回统计字典"""

    scores = [s["score"] for s in students]

    best = max(students, key=lambda s: s["score"])
    worst = min(students, key=lambda s: s["score"])

    return {
        "count": len(students),
        "average": sum(scores) / len(scores),
        "highest": {"name": best["name"], "score": best["score"]},
        "lowest": {"name": worst["name"], "score": worst["score"]},
    }

def filter_by_threshold(students: list[dict], threshold: float) -> list[dict]:
    """筛选分数不低于 threshold 的学生"""

    return [s for s in students if s["score"] >= threshold]

```

这一层做纯计算和逻辑判断。它不知道数据是从 CSV 来的还是数据库来的（它通过参数接收数据），也不知道结果会被打印到控制台还是生成图表。它依赖数据访问层，但不依赖用户交互层。

第三步：用户交互层（管怎么展示）

ui.py

```

from dal import load_students, save_results

from service import calculate_stats, filter_by_threshold

def show_report(filename: str) -> None:
    """从文件加载数据，打印统计报告"""

    students = load_students(filename)

    if not students:
        print("没有学生数据")

        return

    stats = calculate_stats(students)

    print(f"学生总数: {stats['count']}")

    print(f"平均分: {stats['average']:.1f}")

    print(f"最高分: {stats['highest']['name']} ({stats['highest']['score']})")

```

```

print(f"最低分: {stats['lowest']['name']} ({stats['lowest']['score']})")

# 展示优秀学生

excellent = filter_by_threshold(students, 80)

if excellent:

    print(f"\n 优秀学生 (≥80 分) 共 {len(excellent)} 人: ")

    for s in excellent:

        print(f"  {s['name']}: {s['score']}")

if __name__ == "__main__":

    show_report("scores.csv")

```

这一层负责“组装”——它调用下层获取数据、执行计算，然后决定以什么形式展示给用户。它本身不做复杂运算，只做协调和展示。

分层之后的好处：

- 1.想换数据存储方式？只改 `dal.py`。
- 2.想加新的统计指标？只改 `service.py`。
- 3.想改成图形界面输出？只改 `ui.py`。
- 4.每层可以独立测试。你可以单独测 `calculate_stats()` 而不用准备一个 CSV 文件。

这个例子很小，但思想完全适用于更大项目。从现在开始，每当你写一个新程序，先花五分钟想清楚：哪些是数据存取、哪些是核心规则、哪些是用户交互。这个习惯会让你在代码从两百行长到两千行时，依然睡得着觉。

16.4 阅读大型开源项目的方法论

你不可能一辈子只写自己的小项目。迟早，你会需要读懂别人的代码——可能是同事写的，可能是开源库的源码。面对几万行陌生的代码，从哪里下手？

方法一：从外部接口倒推

以 `requests` 库为例。你天天用 `requests.get(url)`。想知道它内部怎么处理重定向的？

- 1.找到暴露给用户的“入口”。`requests.get` 函数定义在 `api.py` 里。
- 2.它调用了 `request("GET", url, ...)`，后者创建了一个 `Session` 对象，调用了 `session.request`。
- 3.在 `sessions.py` 的 `request` 方法里，你会看到一个 `while` 循环，循环条件是 `allow_redirects`，里面每次拿到响应检查状态码，决定是否继续跳转。

跟着一个你熟悉的功能调用栈往下走，边看边用 IDE 的“跳转到定义”（F12）功能。不求逐行理解，只求搞清一个“故事线”。

方法二：先看测试

好的开源项目测试覆盖率高，测试文件本身就是最好的文档。以 `FastAPI` 为例，它的 `tests/` 目录里有几千个测试文件。找一个你感兴趣的功能名，看对应的测试。测试代码展示了库的用法

和预期行为，比文档更精确。

方法三：用 debugger 直接停进去

在你的代码里 import requests，然后在 requests.get 调用处打断点，用 VSCode “单步进入”（Step Into）。程序会带你走进 requests 的源码，你亲眼看到一个请求是怎么一层层走下去的。这种体验比看文档震撼得多。

推荐第一个“阅读目标”：requests 库。它纯 Python、代码量适中（约 4000 行）、结构清晰，是学习大型库设计的绝佳范本。

16.5 持续学习的路径

这是一本带你入门到精通的书，但编程这行没有真正的“学完”。以下是你会继续碰到的领域，以及进入每个领域的第一块垫脚石：

按照本书的路线自然延伸

数据处理：你已经会用 Pandas。下一步可以学可视化（matplotlib/seaborn）和机器学习（scikit-learn）。

Web 开发：FastAPI 搭好后端。前端可以学 HTML/CSS/JavaScript 入门，或者直接上 Vue.js 做单页面应用。

自动化：用 os、shutil、schedule 库写自动化脚本——定时备份文件、批量重命名、自动发送邮件。

数据库：SQLite 够个人项目。多用户场景学 PostgreSQL，高并发学 Redis 高级用法。

进阶 Python 特性

描述符（Descriptor）：理解@property 是怎么实现的。

元类（Metaclass）：类的类，Django 的模型定义就靠它。

C 扩展：用 Cython 或 Rust 给 Python 写底层模块。

并发模式：asyncio 深入（Future、Task、事件循环的内部原理）。

参与开源社区

从小事做起：修复文档错别字，给项目加类型注解。

在 GitHub 上搜索标签 good first issue。

参与翻译文档、回答 Stack Overflow 问题。

你自己写的工具，如果觉得有用，大胆开源到 GitHub。写 README，加 CI，让别人能用起来。最烂的公开仓库也胜过最完美的私藏代码。

18.6 最终实战：一个分层结构的天气预报服务

我们用一个完整的项目来结尾。这个小工具能读取 CSV 数据、做基本统计分析、导出分析报告。

项目结构：

```
data_analyzer/
├── main.py          # 入口，组装流程
├── models.py        # 数据模型（类型定义）
├── dal.py           # 数据访问层：读写文件
├── service.py       # 业务逻辑层：统计分析
├── report.py        # 用户交互层：生成报告
├── test_service.py  # 测试
└── requirements.txt
```

models.py: 定义数据类型

```
from dataclasses import dataclass
from typing import Optional

@dataclass
class Student:
    """学生数据模型"""
    name: str
    score: float
    grade: Optional[str] = None # 评级（后续计算填充）

@dataclass
class StatsResult:
    """统计结果"""
    total: int
    mean: float
    median: float
    highest: Student
    lowest: Student
```

dal.py: 数据读写

```
import csv

from models import Student
from typing import List

def load_students(filepath: str) -> List[Student]:
    """从 CSV 加载学生列表"""
```

```

students = []

with open(filepath, "r", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        students.append(Student(
            name=row["name"],
            score=float(row["score"])
        ))

return students

def save_report(filepath: str, report_text: str) -> None:
    """将报告文本保存为文件"""
    with open(filepath, "w", encoding="utf-8") as f:
        f.write(report_text)

```

service.py: 核心逻辑

```

from statistics import mean, median
from models import Student, StatsResult
from typing import List

def calculate_stats(students: List[Student]) -> StatsResult:
    """计算基本统计量"""
    scores = [s.score for s in students]
    return StatsResult(
        total=len(students),
        mean=mean(scores),
        median=median(scores),
        highest=max(students, key=lambda s: s.score),
        lowest=min(students, key=lambda s: s.score),
    )

def assign_grades(students: List[Student]) -> None:
    """根据分数为学生评级（修改原对象）"""
    for s in students:
        if s.score >= 90:

```



```

        s.grade = "优秀"

    elif s.score >= 80:

        s.grade = "良好"

    elif s.score >= 60:

        s.grade = "及格"

    else:

        s.grade = "不及格"

def filter_by_grade(students: List[Student], grade: str) -> List[Student]:

    """按评级筛选学生"""

    return [s for s in students if s.grade == grade]

```

report.py: 生成报告

```

from dal import load_students, save_report

from service import calculate_stats, assign_grades, filter_by_grade

def generate_report(input_file: str, output_file: str = "report.txt") -> None:

    """从输入文件加载数据，生成分析报告并保存"""

    students = load_students(input_file)

    if not students:

        print("数据为空，无法生成报告")

        return

    assign_grades(students)

    stats = calculate_stats(students)

    lines = [

        "=" * 40,

        "学生成绩分析报告",

        "=" * 40,

        f"学生总数: {stats.total}",

        f"平均分: {stats.mean:.2f}",

        f"中位数: {stats.median:.1f}",

        f"最高分: {stats.highest.name} ({stats.highest.score})",

        f"最低分: {stats.lowest.name} ({stats.lowest.score})",

        ""
    ]

```

```

    "评级分布: ",

]

for grade in ["优秀", "良好", "及格", "不及格"]:

    count = len(filter_by_grade(students, grade))

    lines.append(f" {grade}: {count} 人")

report = "\n".join(lines)

print(report)

save_report(output_file, report)

print(f"\n 报告已保存至 {output_file}")

```

main.py: 入口

```

from report import generate_report

import sys

def main():

    if len(sys.argv) < 2:

        print("用法: python main.py <数据文件.csv>")

        sys.exit(1)

    generate_report(sys.argv[1])

if __name__ == "__main__":

    main()

```

这个项目虽然小，但五脏俱全——分层清晰、依赖方向单一、测试友好（你可以单独导入 `service.calculate_stats` 做测试）、可扩展（想加柱状图？加一个 `chart.py` 层即可，不动已有代码）。

这是你书中学到的所有东西的会师：面向对象（数据模型）、函数（统计计算）、文件操作（CSV 读写）、错误处理、分层设计、命令行入口。

16.7 全书总结与你的下一行代码

我们花了十六章，走了一条很长的路。回忆一下：

你从一个连 `print(2+3)` 都小心翼翼的新手，变成了能写函数、设计类、组织模块、抓取网页、分析数据、搭建 Web 服务，并把它部署到云上的程序员。更重要的是，你建立了程序运行的思维模型——变量是标签、对象是积木、函数是契约、服务是永远在线的等待。

你现在能做的事：

自动化日常工作中重复的数据处理

爬取并分析公开数据

把 Python 脚本封装成他人可用的工具

参与到真实世界的开源项目

你不能做的事（至少现在还不能，但有了前进方向）：

写出 Google 搜索那样的搜索引擎（需要分布式系统知识）

写出 World of Warcraft 这样的游戏（需要图形引擎和 C++ 底层）

训练出 GPT 级别的模型（需要深入研究机器学习）

但你站在了门槛上。从这扇门出去，无论是数据科学、后端工程、运维自动化，还是后续
的算法、系统设计，Python 都将是手中称手的工具。

最后一个建议：不要停止写代码。选一个小项目，把它做完、做好、部署上线、加入测试、
写说明文档、开源出来。你写的每一个真实项目，都胜过刷一百道练习题。

有一句 Python 社区流传甚广的话，我愿意用它来为这本书收尾：

"Come for the language, stay for the community."

因为一门语言而来，因为这个温暖的社区而留下。

现在，打开你的 VSCode。新建一个 .py 文件。写你的下一个项目的第一行代码。

```
# 从这里开始
```

练习

最后的练习没有标准答案，它们是你接下来的方向：

1. 回头看之前写的所有代码，挑一个你最得意的函数或类，加上完整的类型注解、文档字符串和 pytest 测试。

2. 将你的一个已有脚本，重构成分层结构（数据层、业务层、用户交互层分离），观察改动带来的清晰度变化。

3. 打开 GitHub，克隆 requests 或 Flask 仓库，找到 get 函数或 app.route 的源码入口，跟着调用栈阅读 100 行代码。

4. 给自己写一个三个月计划：要学什么（比如机器学习/前端/Vue）、要做什么项目（比如一个博客 API/数据看板/小程序后端），然后开始第一步。

5. 把你的一个项目推到 GitHub 上，写好 README 说明怎么用。分享链接给一个朋友，让他照着你的说明跑起来，修好他遇到的每一个问题。这就是真实世界的软件开发。